

華東理工大學

计算机图形学

2023年12月

奉贤校区



華東理工大學

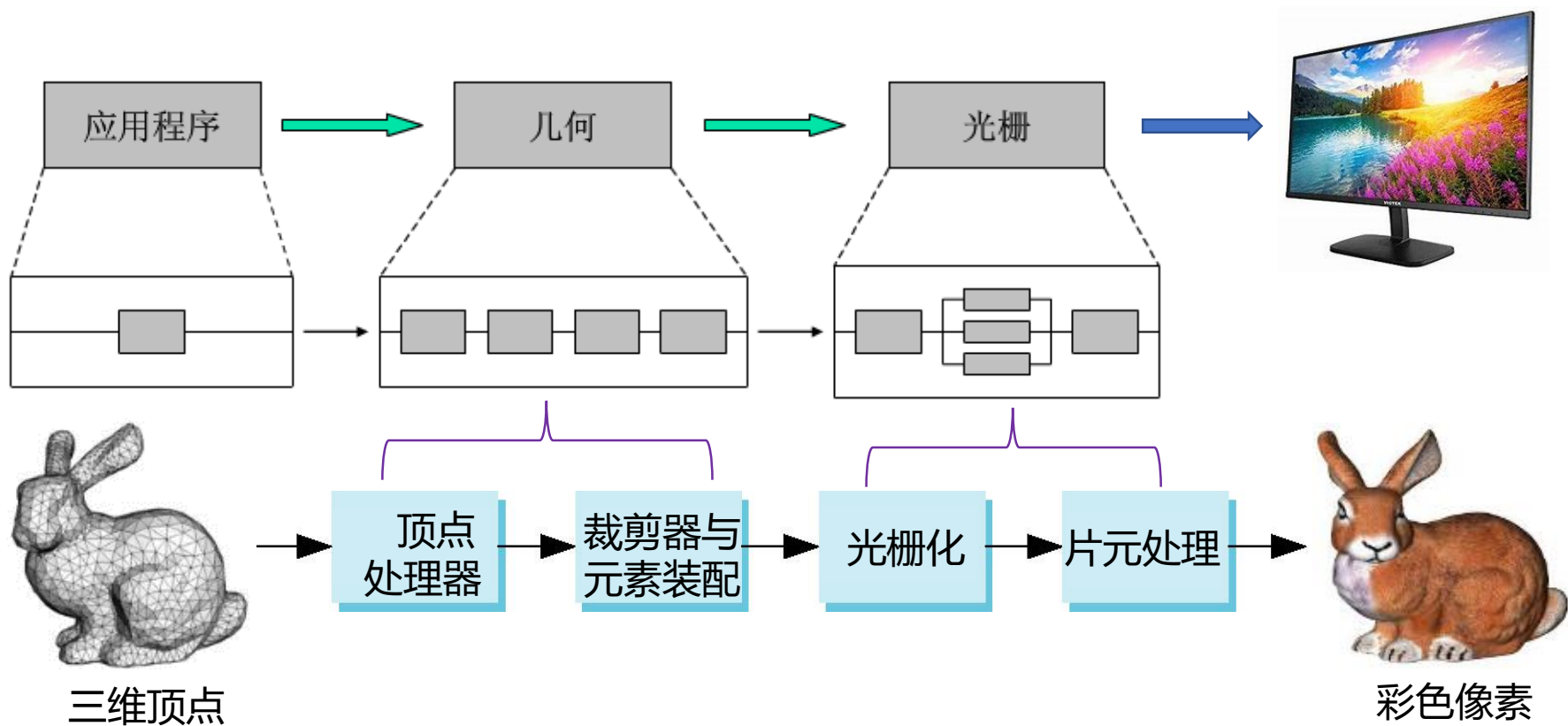


10

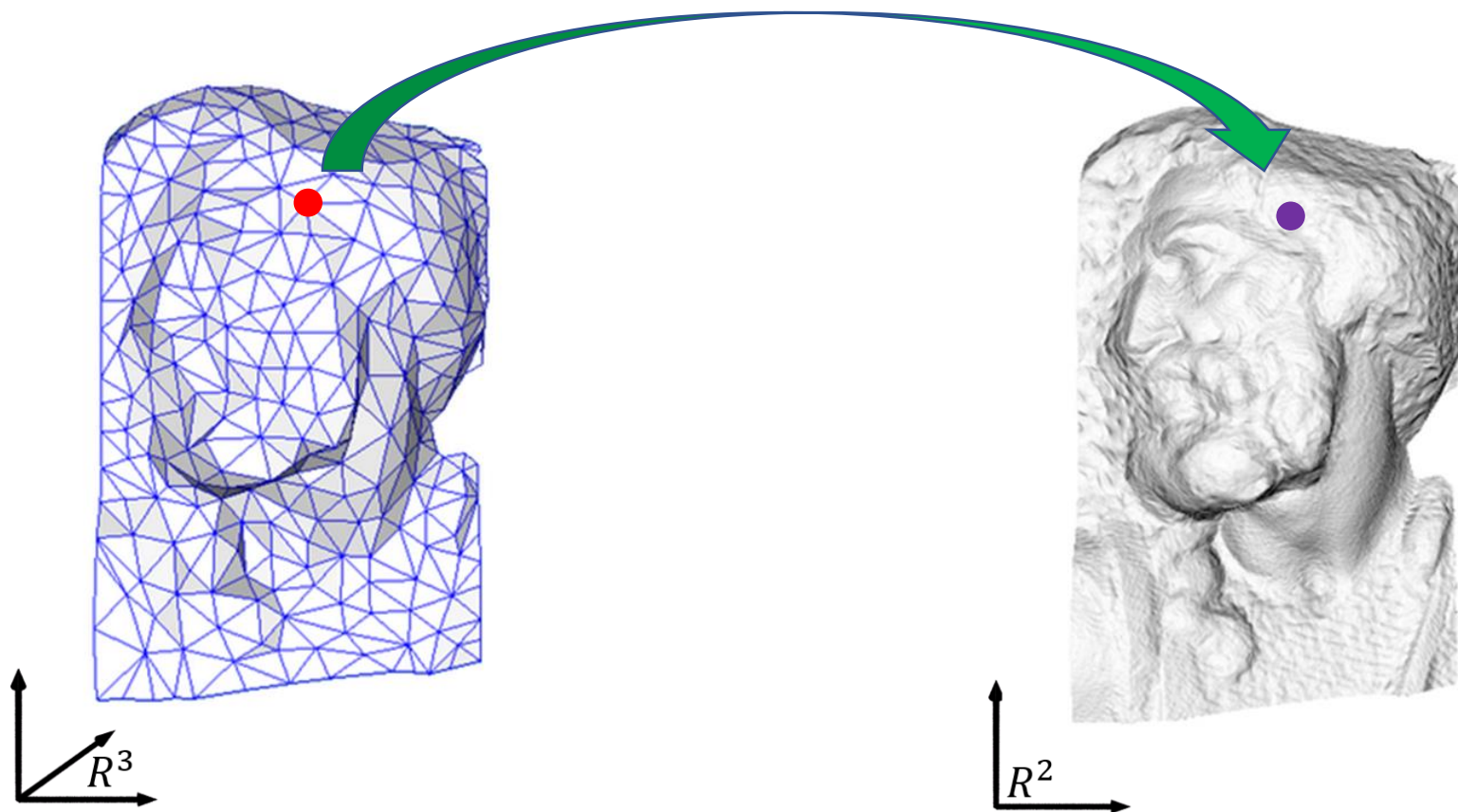
真实感图形绘制

Photorealistic Rendering

渲染管线：渲染计算的流水线

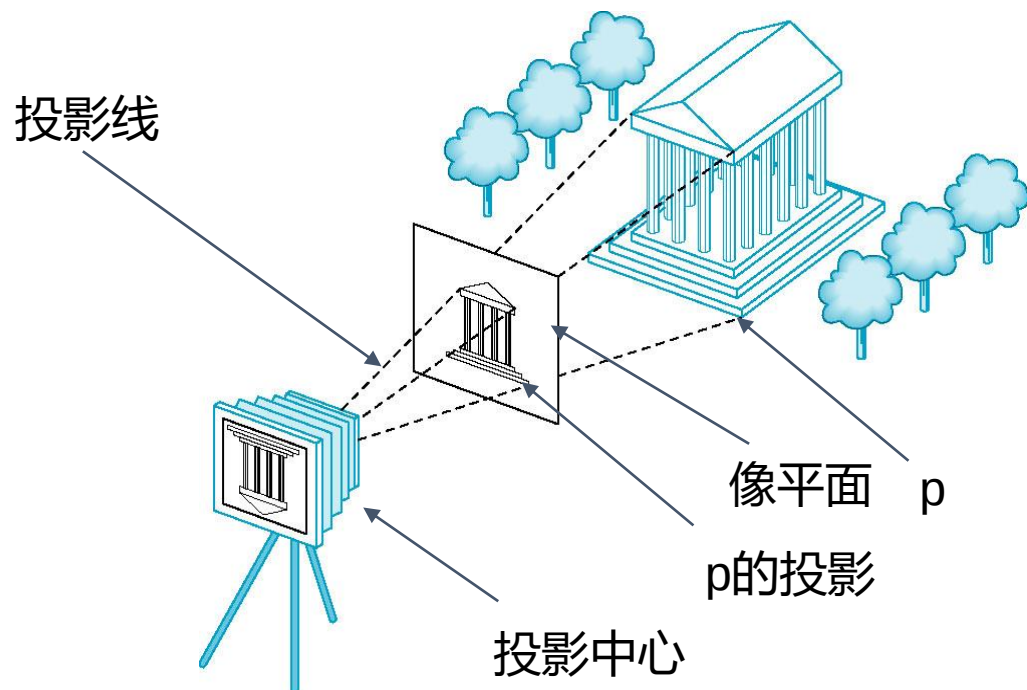


过程1：几何变换(逐顶点计算屏幕坐标)



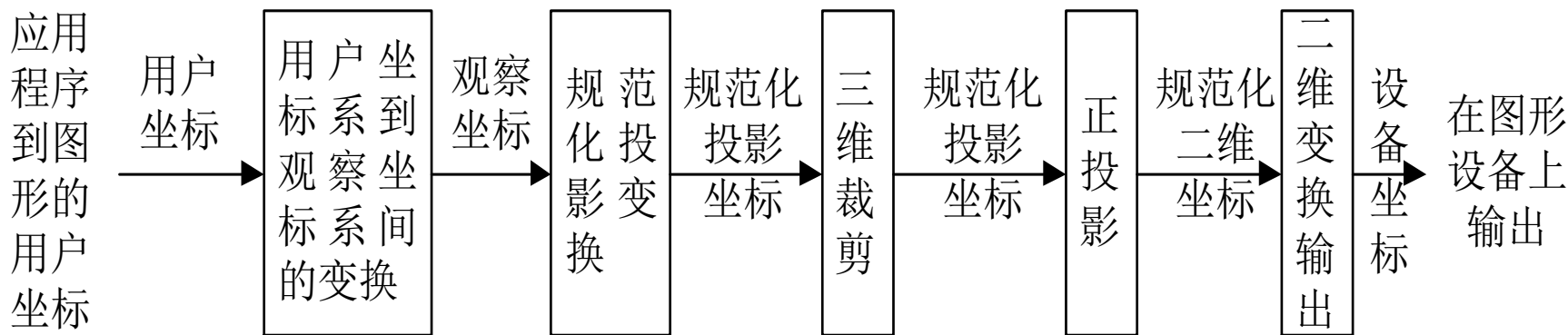
成像模型

- 3D场景在虚拟相机（眼睛）下的投影
- 应用程序指定
 - 3D模型
 - 相机参数
 - 位置
 - 朝向
 - 焦距
 - 视角

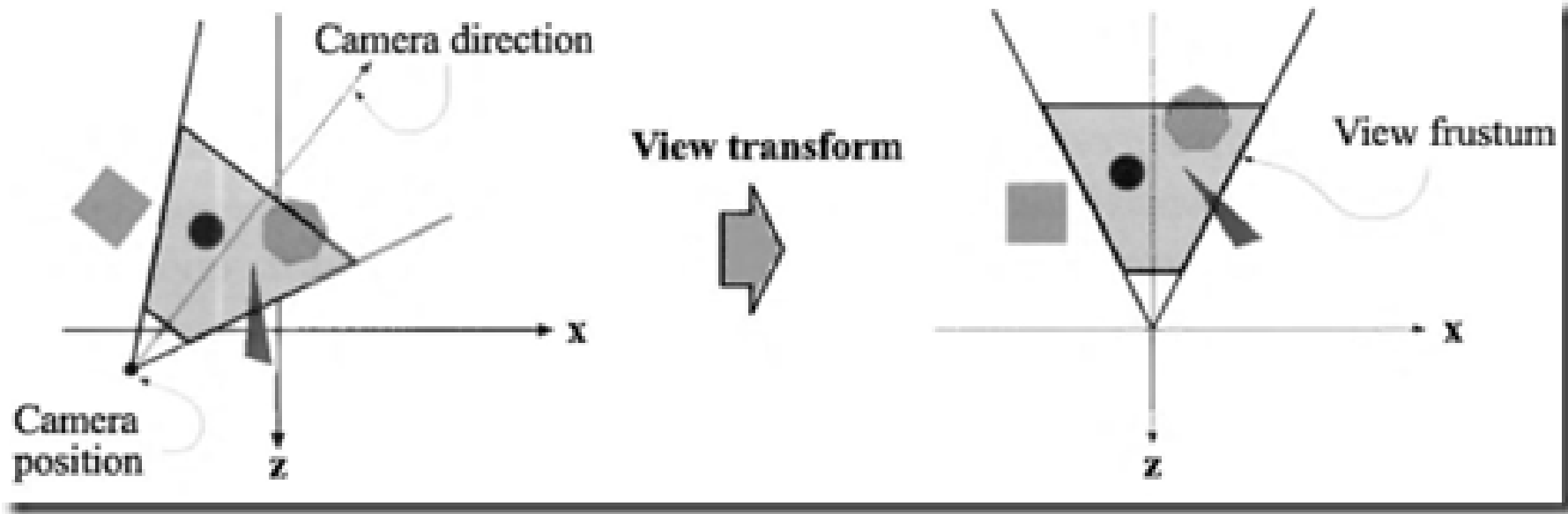


顶点处理

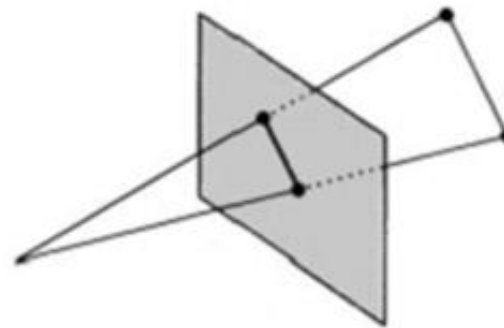
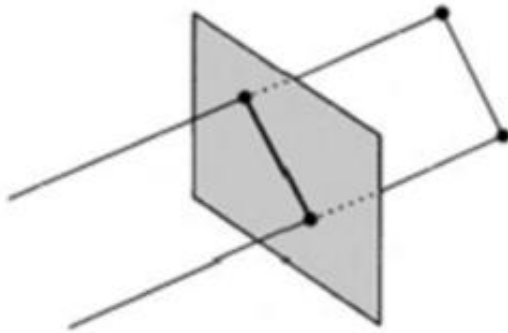
- 流水线中大部分工作是把对象在一个坐标系中表示转化为另一坐标系中的表示：
 - 世界坐标系
 - 照相机(眼睛)坐标系
 - 屏幕坐标系
- 坐标的每个变换相当于一次**矩阵乘法**
 - 最终的顶点变换为多个矩阵的乘法：MVP
 - 窗口变换



模型及视图变换

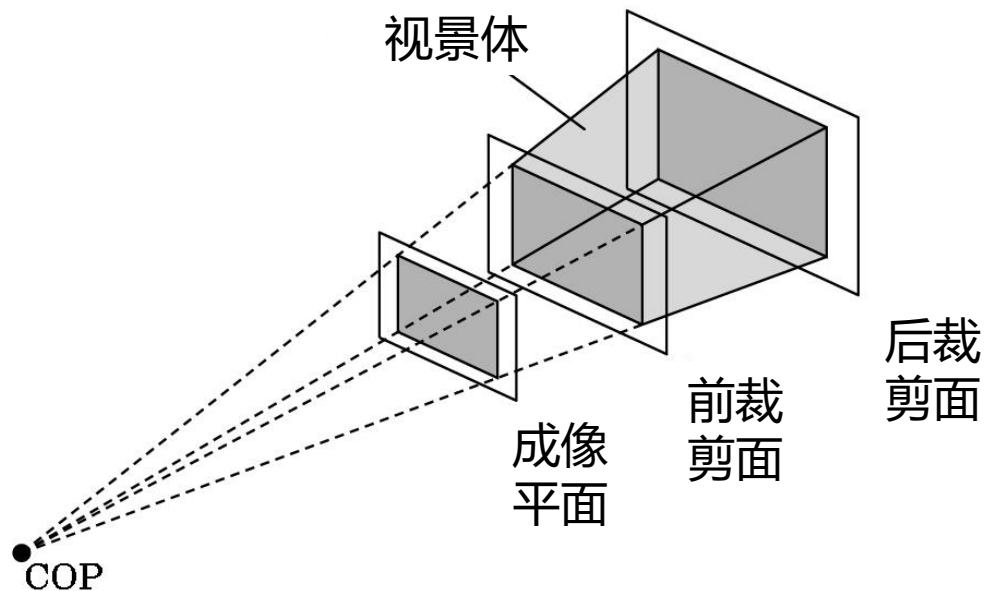
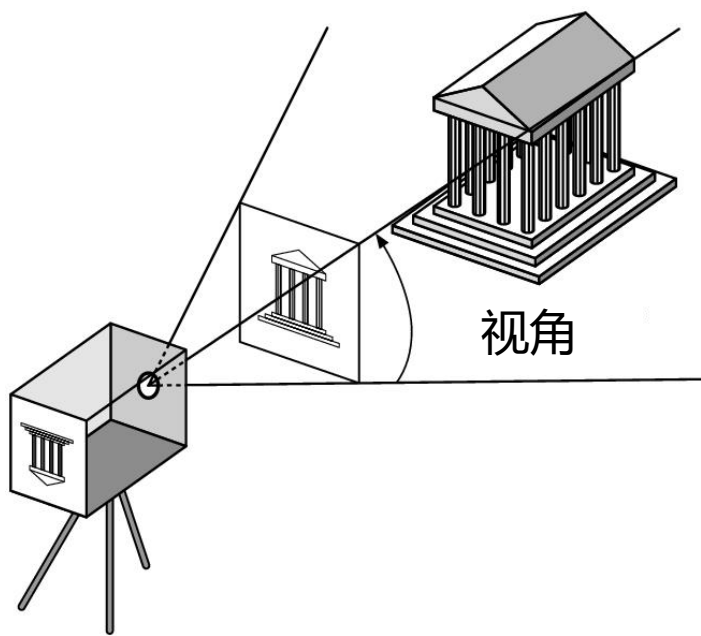


投影变换

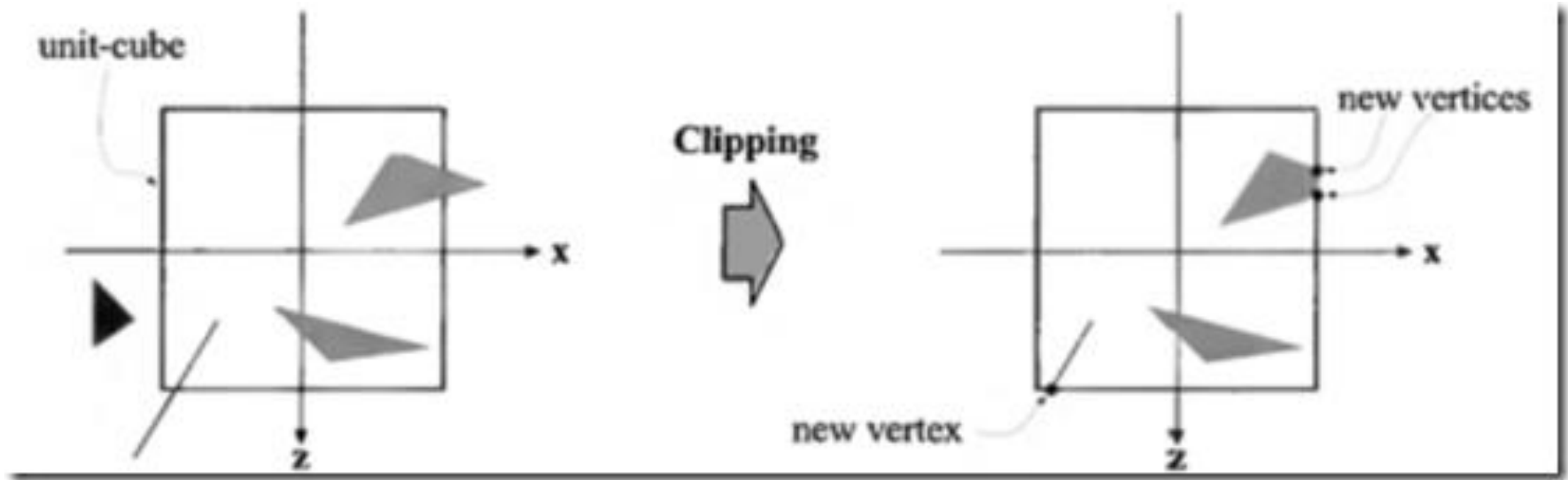


裁剪：视景体裁剪

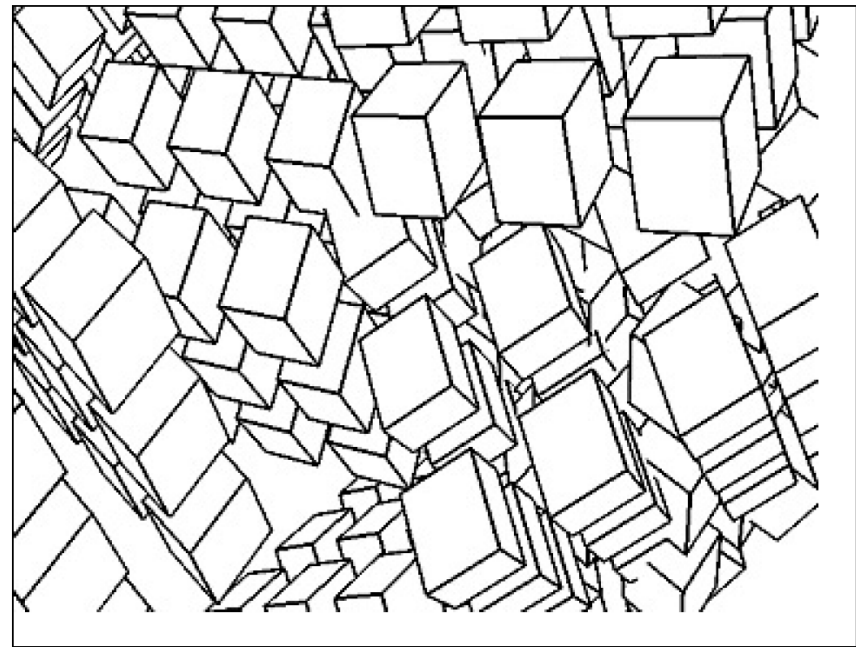
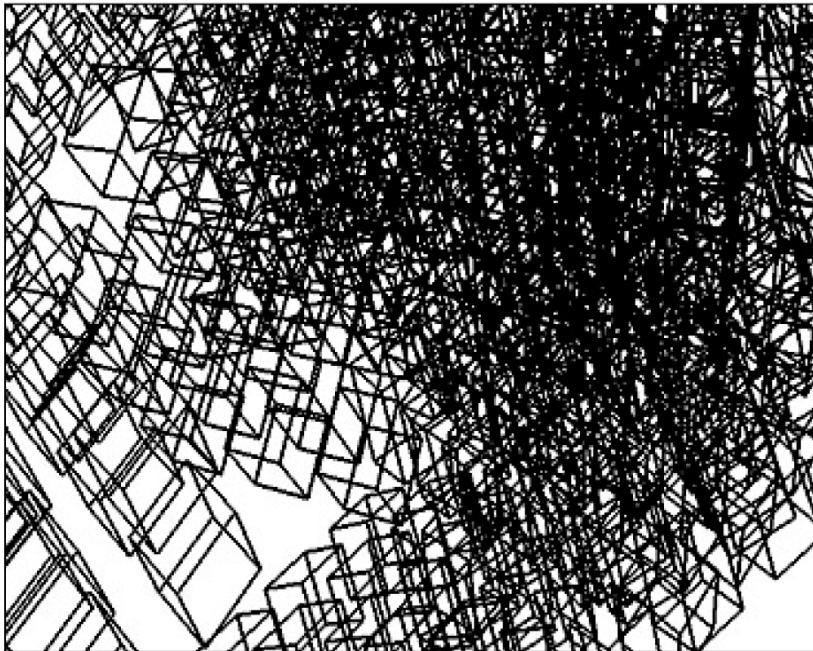
- 超出可视范围的几何元素需要裁剪掉，不参加后面的计算
 - 视景体范围



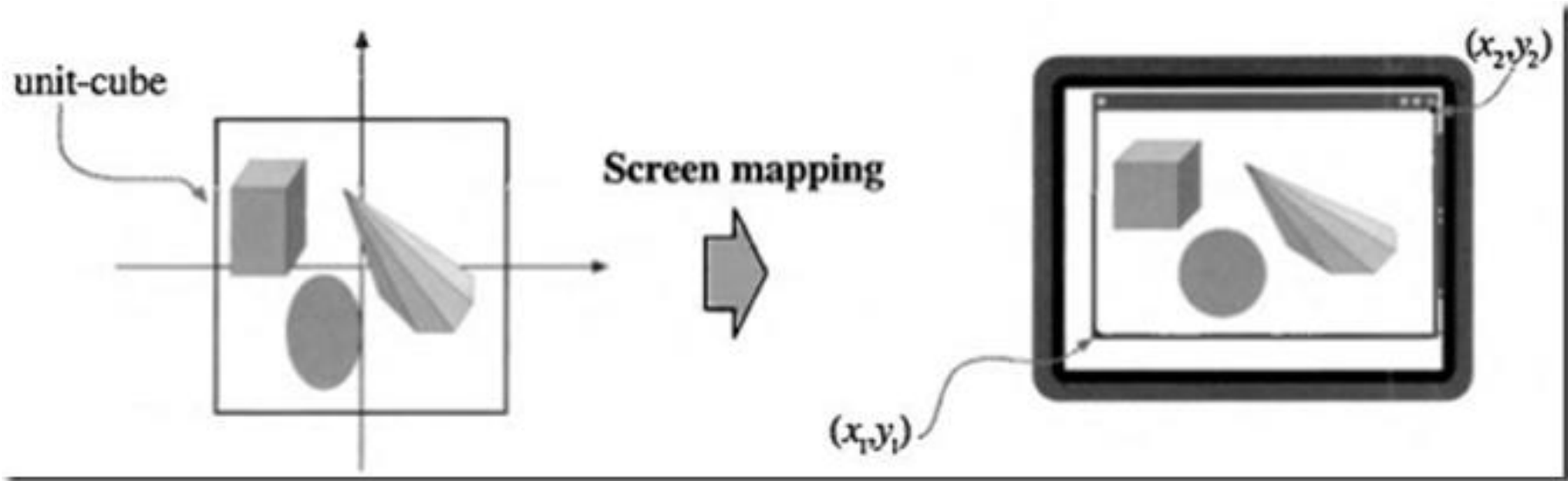
裁剪：窗口裁剪



消隐：消除隐藏面



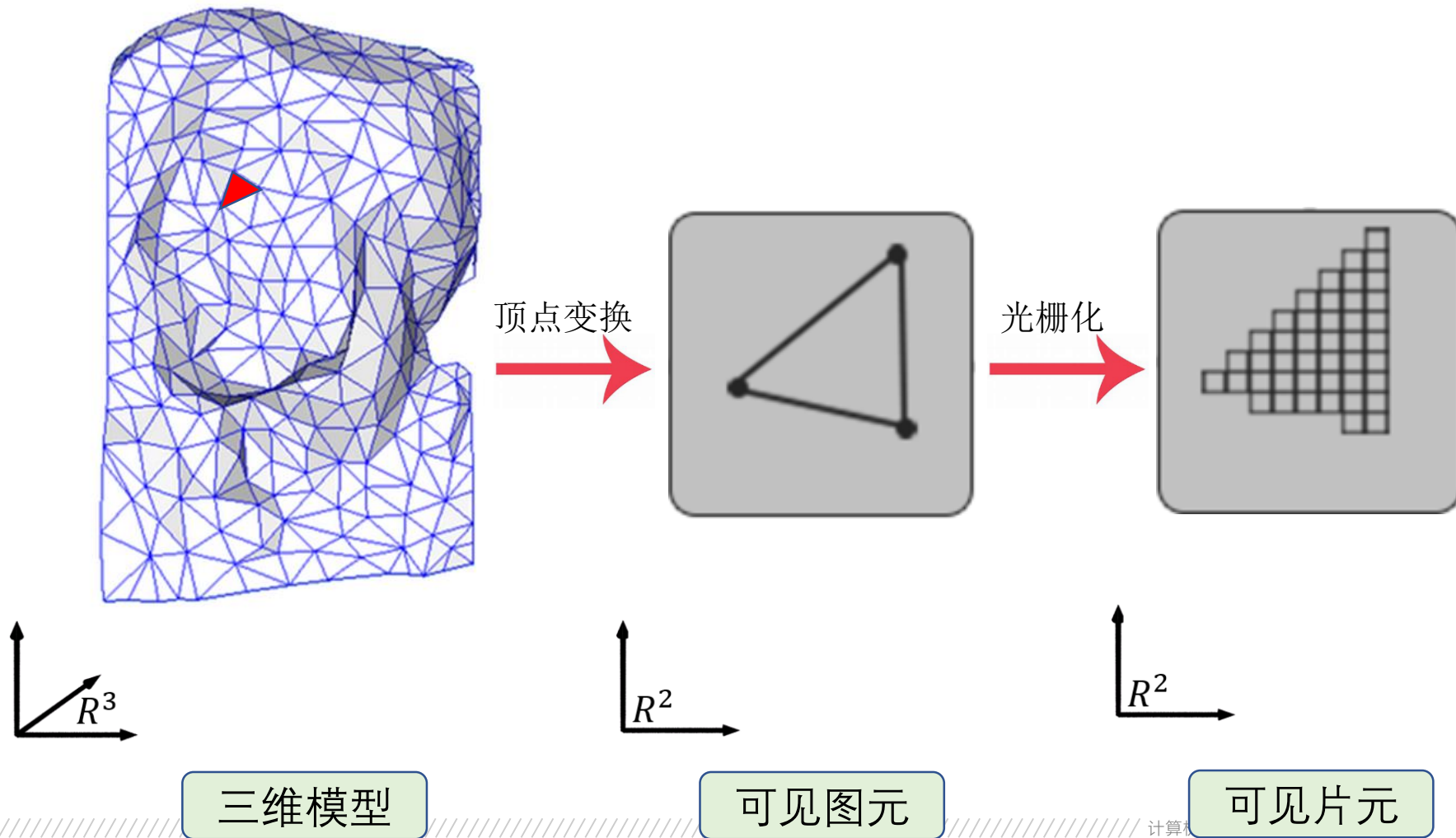
屏幕映射变换



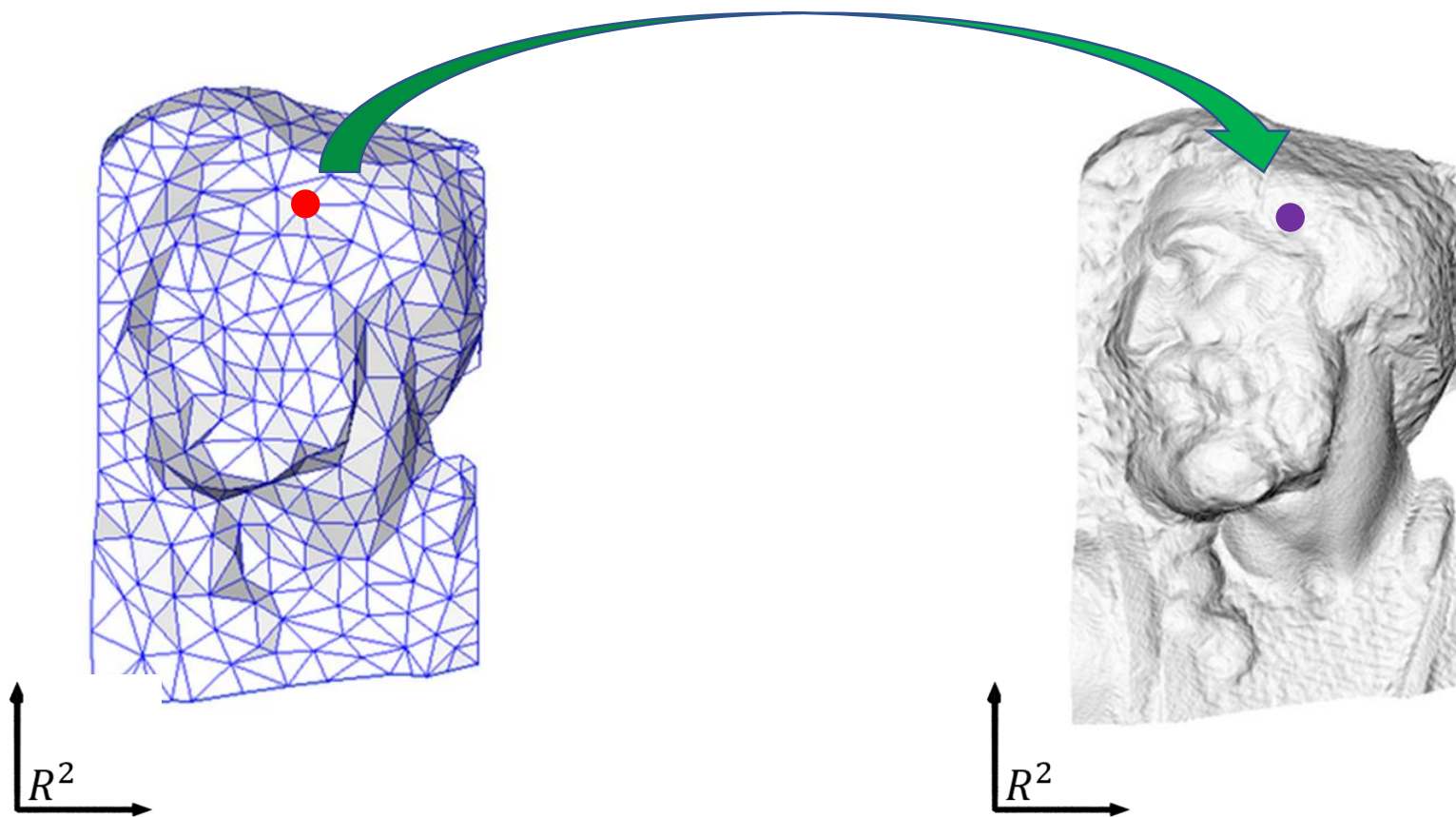
光栅化：找到所有片元

- 如果一个对象不被裁掉，那么在帧缓冲区中相应的像素就必须被赋予颜色
- 光栅化程序为每个对象生成一组片段
- 片段是“潜在的像素”
 - 在帧缓冲区中有一个位置
 - 具有颜色和深度属性
- 光栅化程序在对象上对顶点属性进行插值（重心坐标）

过程1：几何变换(逐顶点计算屏幕坐标)



过程2：像素着色（逐片元计算颜色）



着色：让3D物体更有空间感

- 光：产生物体的不同部分的明暗变化
- 需模拟光对顶点的明暗（颜色）计算



主要内容

- 基本概念
- 简单光照模型
- 基于简单光照模型的多边形绘制
- 模拟景物表面细节（纹理映射）
- 整体光照模型
- 光线追踪

- **真实感图形绘制**：通过综合利用数学、物理学、计算机以及心理学等知识在计算机图形输出设备上绘制出能够以假乱真的美丽景象。
- **光强（度）**：描述物体表面朝某方向辐射光的颜色，它既能表示光能大小又能表示其色彩组成的物理量。

- **光照模型** (Illumination model) , 也称**明暗模型**, 主要用于物体表面某点处的光强度计算。
 - 简单的光照模型
 - 复杂的光照模型

□ 真实感图形绘制过程

根据假定的光照条件和景物外观因素，依据一定的光照模型，**计算可见面**投射到观察者**眼中的光强度大小**，并将它转换成适合图形设备的**颜色值**，生成投影画面上每一个像素的光强度，使观察者产生身临其境的感觉。

基本概念

□ 真实感图形绘制步骤

- 在计算机中进行场景造型;
- 进行取景变换和透视变换;
- 进行消隐处理;
- 进行真实感图形绘制。

简单光照模型

1

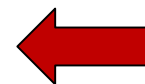
简单光照模型

2

光线衰减

3

颜色处理



简单光照模型

- 简单光照模型中只考虑反射光的作用。
- 反射光由环境光、漫反射光和镜面反射光三部分组成。

环境光 (Background Light)

- 特点：照射在物体上的光来自周围各个方向，又均匀地向各个方向反射。
- P点对环境光的反射强度为

$$I_e = I_a K_a$$

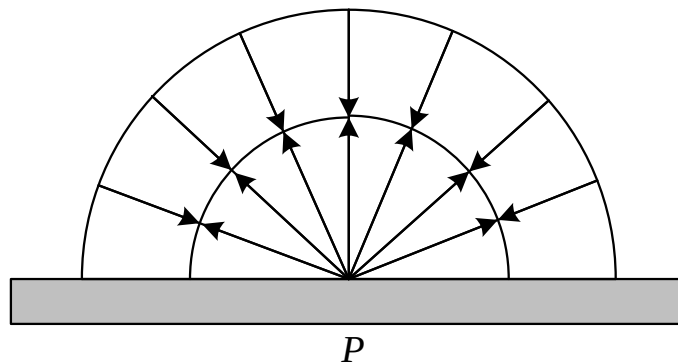


图10.1 环境光的反射

漫反射光 (Diffuse Reflection)

- 一个粗糙的、无光泽的表面呈现为**漫反射**。
- **特点**：光源来自一个方向，反射光均匀地射向各个方向。
- 由Lambert余弦定理可得点P处漫反射光的强度为：

$$I_d = I_p K_d \cos \theta, \theta \in [0, \frac{\pi}{2}]$$

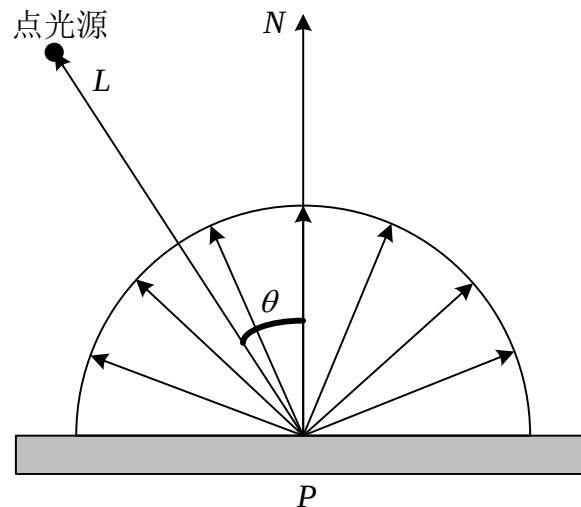


图10.2 漫反射

漫反射光 (Diffuse Reflection)

□ 若 L 和 N 都已规格化为单位矢量，则有

$$I_d = I_p K_d (\mathbf{L} \cdot \mathbf{N})$$

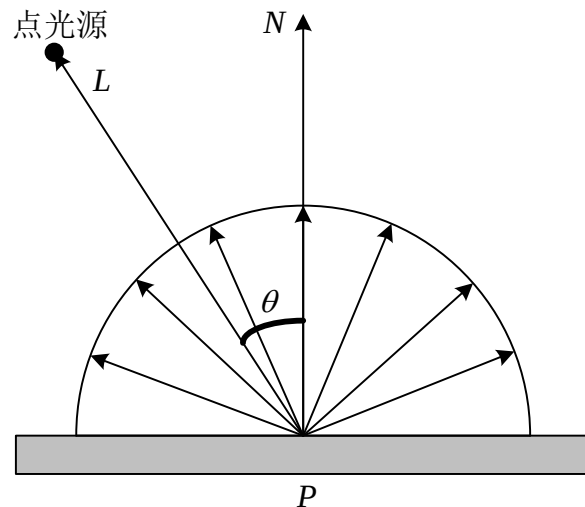


图10.2 漫反射

漫反射光 (Diffuse Reflection)

□ 对于彩色

$$I_p = (I_{pR}, I_{pG}, I_{pB})$$

$$I_{dR} = I_{pR} K_{dR} (L \cdot N)$$

$$I_{dG} = I_{pG} K_{dG} (L \cdot N)$$

$$I_{dB} = I_{pB} K_{dB} (L \cdot N)$$

□ 对于多个漫反射光源

$$I_d = \sum_{i=1}^n I_{p,i} K_d (L_i \cdot N)$$

镜面反射光 (specular reflection)

- 镜面反射遵循反射定律，入射光和反射光分别位于表面法矢的两侧。
- 如果观察者正好处在P点的镜面反射方向上，就会看到一个比周围亮得多的高光点。

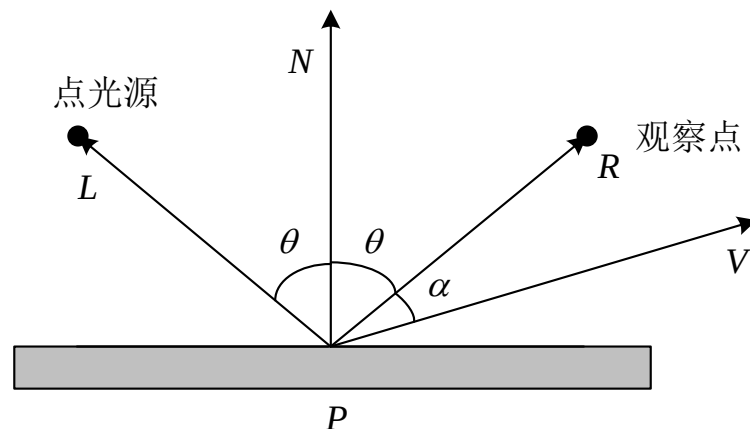


图10.3 镜面反射

镜面反射光

□ 镜面反射情况由Phong模型给出：

$$I_s = I_p K_s \cos^n \alpha$$

□ 若R和V已规格化为单位矢量，则：

$$I_s = I_p K_s (R \cdot V)^n$$

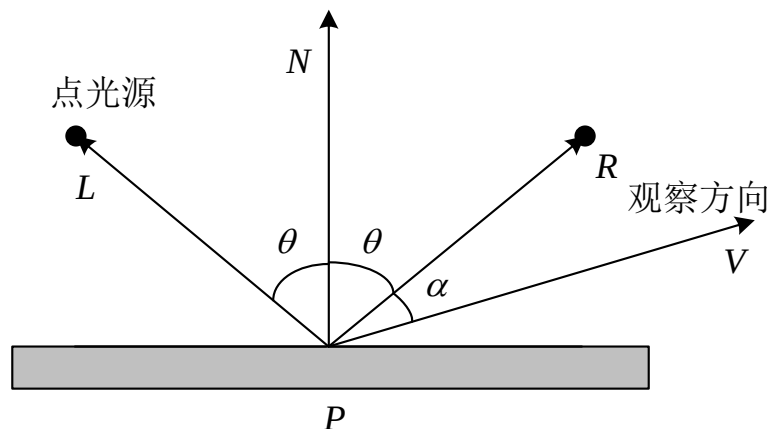


图10.3 镜面反射

物体表面光强计算

- 从视点观察到物体上任一点P处的光强度I应为环境光反射光强度 I_e 、漫反射光强度 I_d 以及镜面反射光的光强度 I_s 的总和：

$$\begin{aligned} I &= I_e + I_d + I_s \\ &= I_a K_a + I_p K_d (L \cdot N) + I_p K_s (R \cdot V)^n \end{aligned}$$

简单光照模型

1

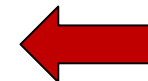
简单光照模型

2

光线衰减

3

颜色处理



- 光在传播的过程中，其能量会发生衰减。光照模型中必须考虑光强衰减，否则会影响生成图形的真实效果。
- 光强的衰减可以采用常数衰减、一次函数衰减和二次函数衰减等。

□ 常用的二次衰减函数

$$f(d) = \min(1, \frac{1}{c_0 + c_1 d + c_2 d^2})$$

$$I = I_e + I_d + I_s$$

$$= I_a K_a + \sum_{i=1}^n f(d_i) I_{p,i} K_d (L_i \cdot N) + \sum_{i=1}^n f(d_i) I_{p,i} K_s (H_i \cdot N)^n$$

Phong模型示例



理想漫反射

+



环境光

+

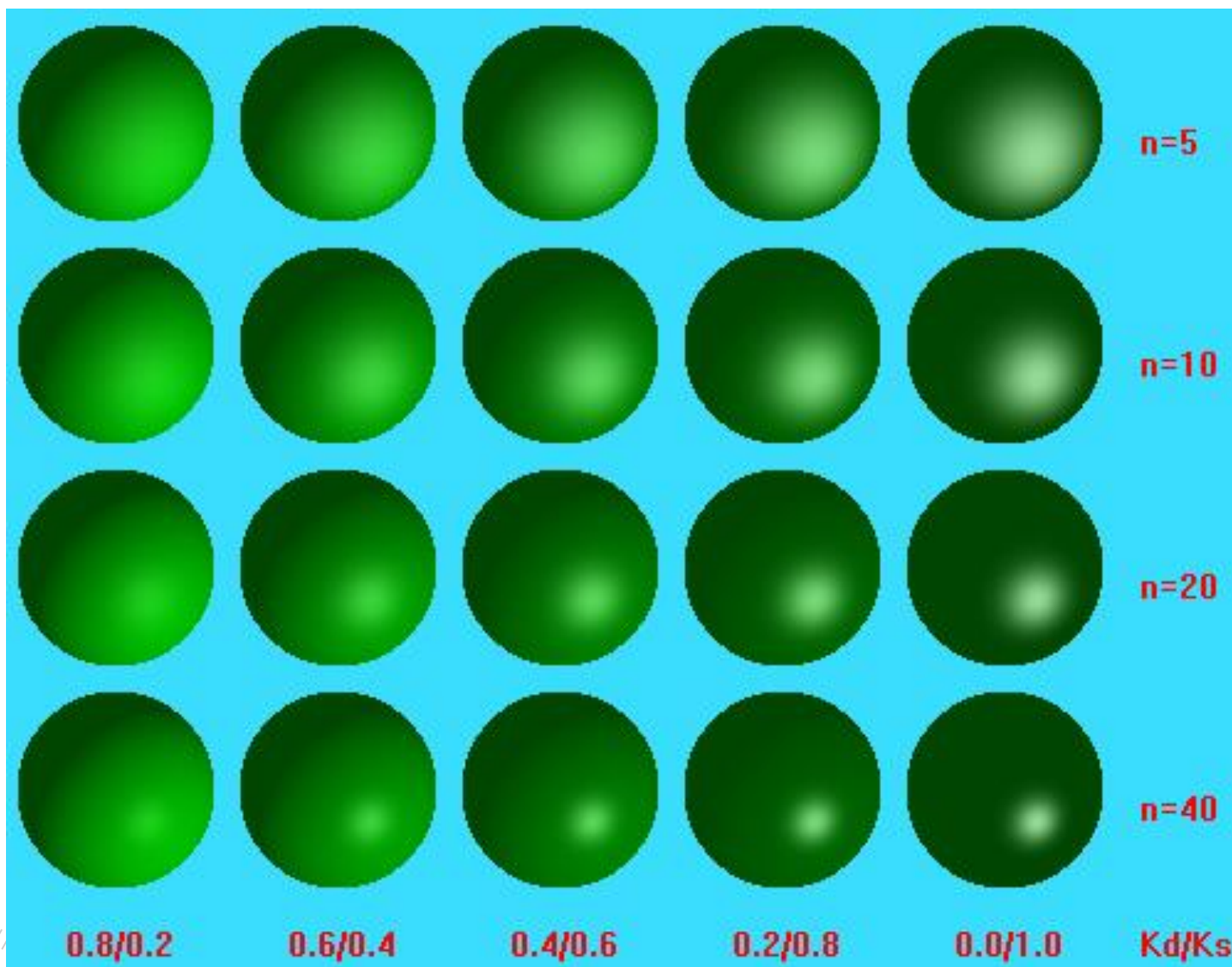


镜面反射

=



Phong模型示例



简单光照模型

1

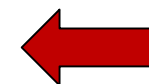
简单光照模型

2

光线衰减

3

颜色处理



- **选择颜色模型 (color model)**
 - **面向硬件的颜色模型: RGB、CYM**
 - **面向视觉感知的颜色模型: HSI**
- **为颜色分量指定光照模型**

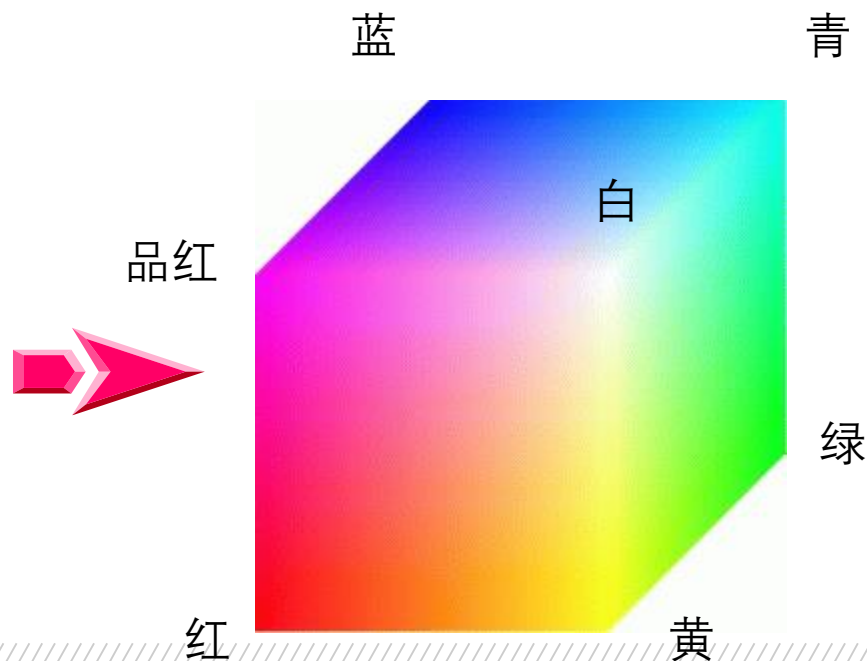
常用颜色模型

- 1) RGB颜色模型
- 2) CMY颜色模型
- 3) HSV颜色模型
- 4) HSI颜色模型

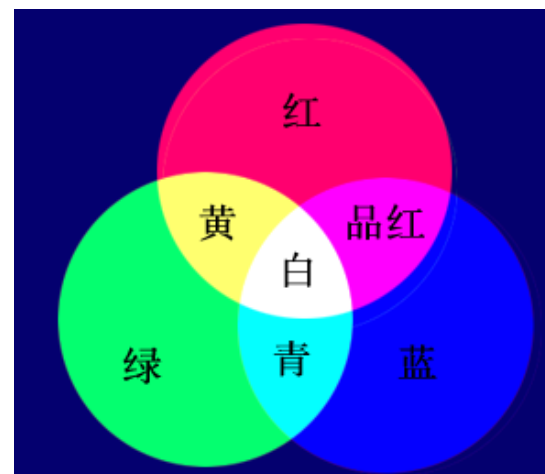
1) RGB颜色模型

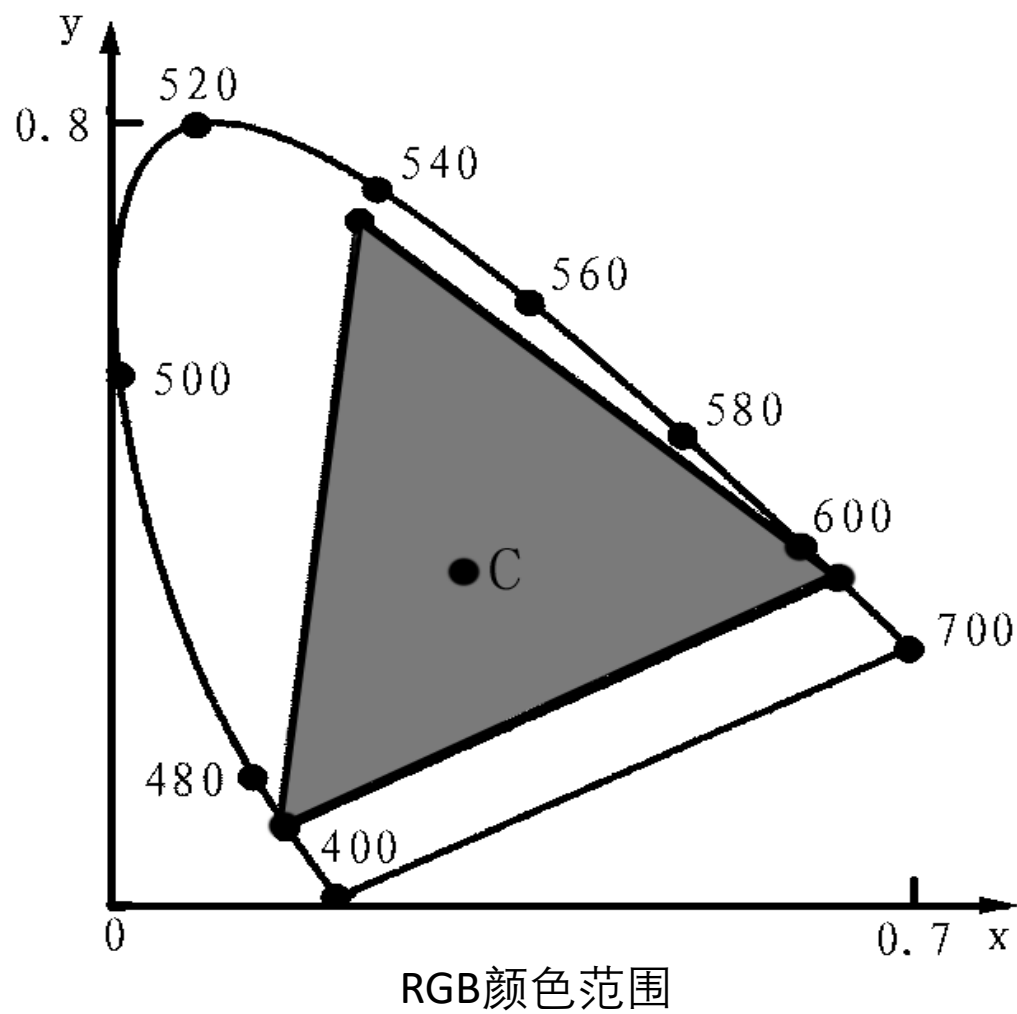
国际照明委员会选择红色（波长 $\lambda=700.00\text{nm}$ ）、绿色（波长 $\lambda=546.1\text{nm}$ ）、蓝色（波长 $\lambda=435.8\text{nm}$ ）三种单色光作为表色系统的三基色

- 通常使用于彩色光栅图形显示设备中；
- 真实感图形学中的主要的颜色模型
- 采用三维直角坐标系
- RGB立方体



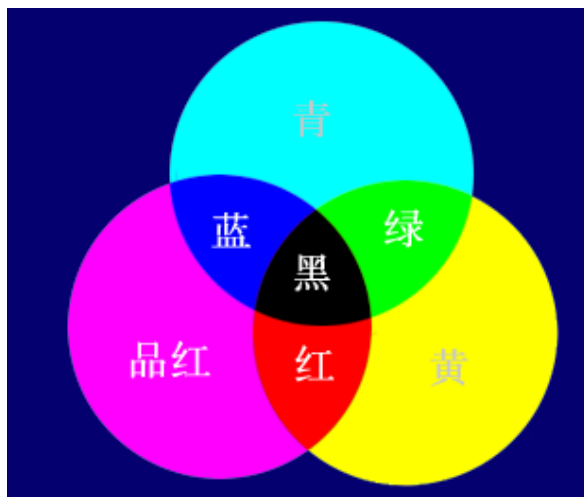
三原色混合效果

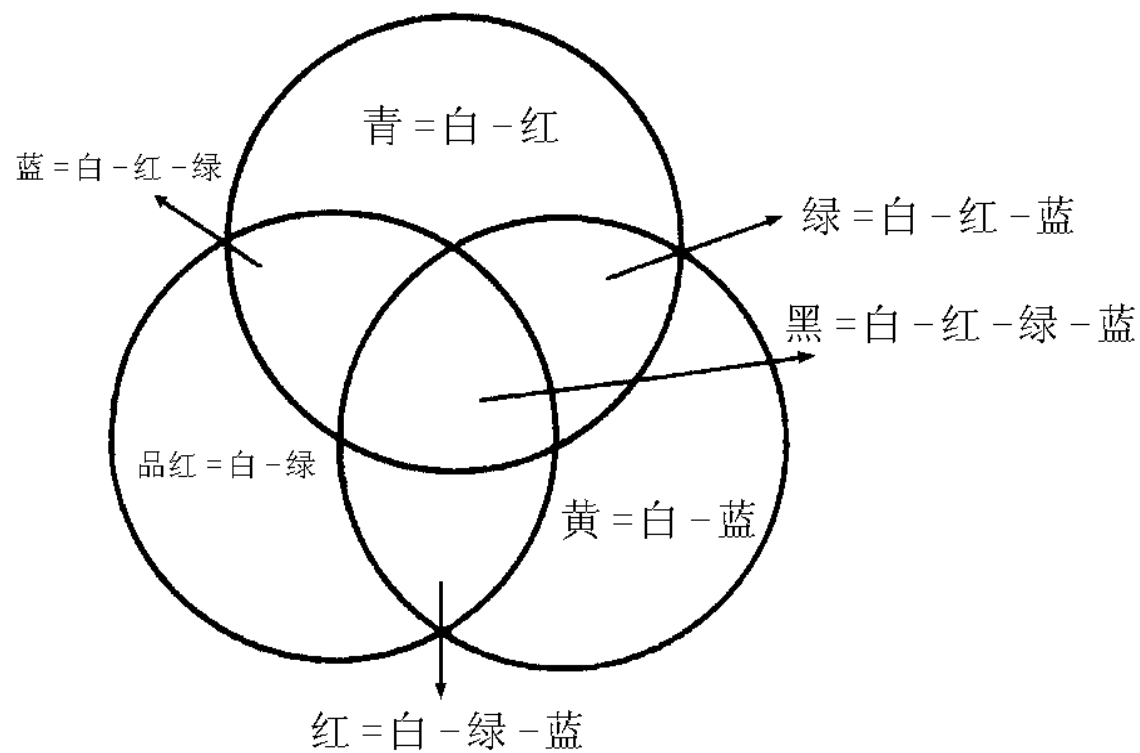




2) CMY颜色模型

以品红、青、黄 (Cyan, Magenta, Yellow) 作为三基色所构成的颜色模型也是一种常用的颜色表示系统。它是一种减色系统。CMY减色系统和RGB加色系统颜色互为补色。所谓某颜色的补色是从白色中减去这种颜色后所得的颜色。品红是绿色的补色，青色是红色的补色，黄色是蓝色的补色。即相加系统的补色就是相减系统的基色 ($R+G=黄$, $G+B=青$, $R+B=品红$)。

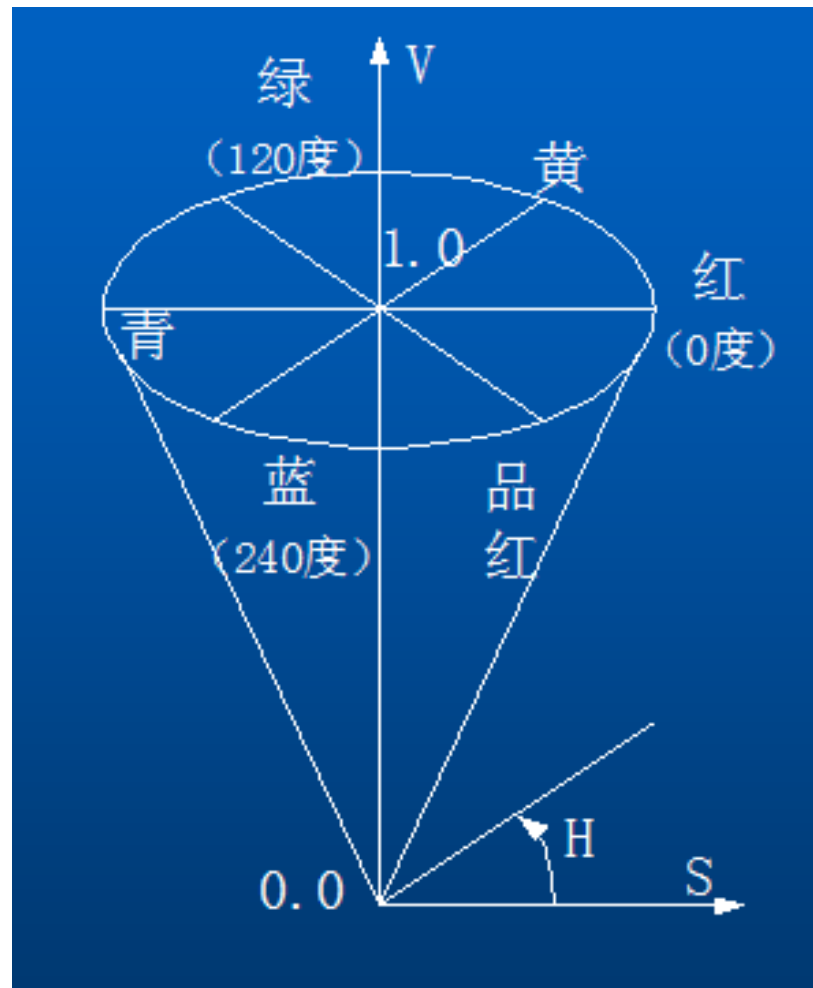




CMY和RGB的关系

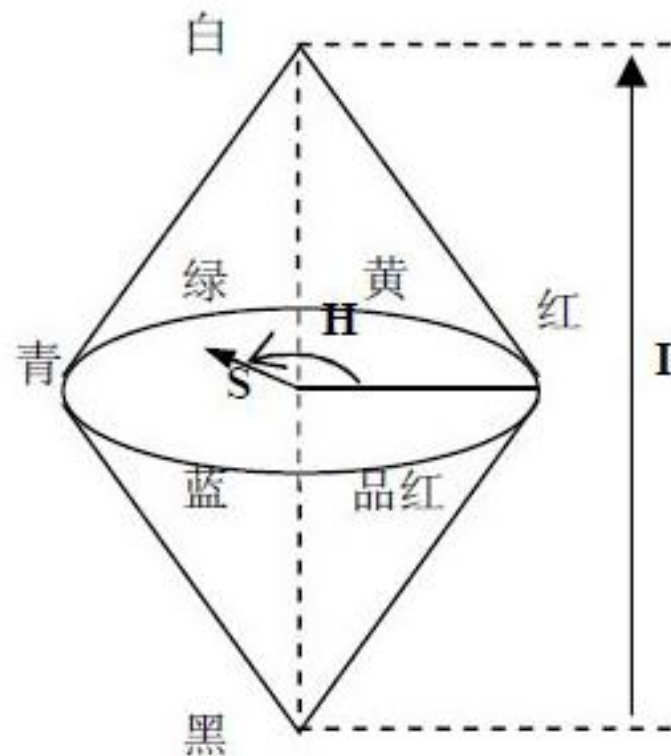
3) HSV颜色模型

- HSV（色度、饱和度、纯度）颜色模型是面向用户的
- 对应圆柱坐标系的圆锥形子集
- 圆锥的顶面对应于 $V=1$
- 色彩 H 由绕 V 轴的旋转角给定
- 饱和度 S 取值从0到1，
- 由圆心向圆周过渡



4) HSI颜色模型

- HSI（色度、饱和度、Intensity（强度））颜色模型是面向用户的
- 对应双锥体表示



色调	饱和度	亮度
$H = \begin{cases} \theta, & G \geq B \\ 2\pi - \theta, & G < B \end{cases}$ $\text{where } \theta = \cos^{-1} \left(\frac{(R-G)+(R-B)}{2\sqrt{(R-G)^2 + (R-B)(G-B)}} \right)$	$S = 1 - \frac{3 \min(R, G, B)}{R + G + B}$	$I = \frac{R + G + B}{3}$

□ 以RGB颜色模型为例

- 环境光强度: $I_a = (I_{aR}, I_{aG}, I_{aB})$
- 入射光强度: $I_p = (I_{pR}, I_{pG}, I_{pB})$
- 环境光反射系数: $K_a = (K_{aR}, K_{aG}, K_{aB})$
- 漫反射系数: $K_d = (K_{dR}, K_{dG}, K_{dB})$
- 镜面反射系数: $K_s = (K_{sR}, K_{sG}, K_{sB})$

□ 光强计算公式:

$$I_R = I_{aR}K_{aR} + \sum_{i=1}^n f(d_i)I_{pR,i}K_{dR}(L_i \cdot N) + \sum_{i=1}^n f(d_i)I_{pR,i}K_{sR}(H_i \cdot N)^n$$

$$I_G = I_{aG}K_{aG} + \sum_{i=1}^n f(d_i)I_{pG,i}K_{dG}(L_i \cdot N) + \sum_{i=1}^n f(d_i)I_{pG,i}K_{sG}(H_i \cdot N)^n$$

$$I_B = I_{aB}K_{aB} + \sum_{i=1}^n f(d_i)I_{pB,i}K_{dB}(L_i \cdot N) + \sum_{i=1}^n f(d_i)I_{pB,i}K_{sB}(H_i \cdot N)^n$$

基于简单光照模型的多边形绘制

1

恒定光强

2

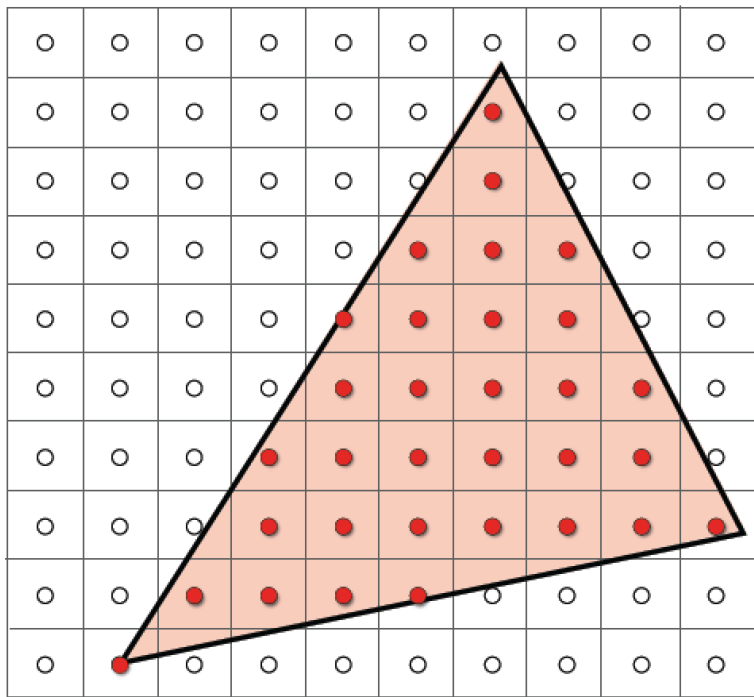
Gouraud明暗处理

3

Phong明暗处理

基于简单光照模型的多边形绘制

非顶点的像素的颜色？



由顶点处的信息插值得到！

□ 只用一种颜色绘制整个多边形

- 光源在无穷远处，则多边形上所有点的 $L \cdot N$ 为常数，衰减函数也是一个常数。
- 视点在无穷远处，则多边形上所有点的 $V \cdot R$ 为常数。
- 多边形是景物表面的精确表示，即不是一个含曲线面景物的近似表示。

- **Gouraud明暗处理方法，又称为亮度插值明暗处理，它通过对多边形顶点颜色进行线性插值来绘制其内部各点（计算光强），其步骤为：**
 - 计算每个多边形顶点处的平均单位法向量；
 - 对每个顶点根据简单光照模型来计算其光强；
 - 在多边形表面上将顶点强度进行线性插值。

Gouraud 明暗处理

□ 双线性插值方法

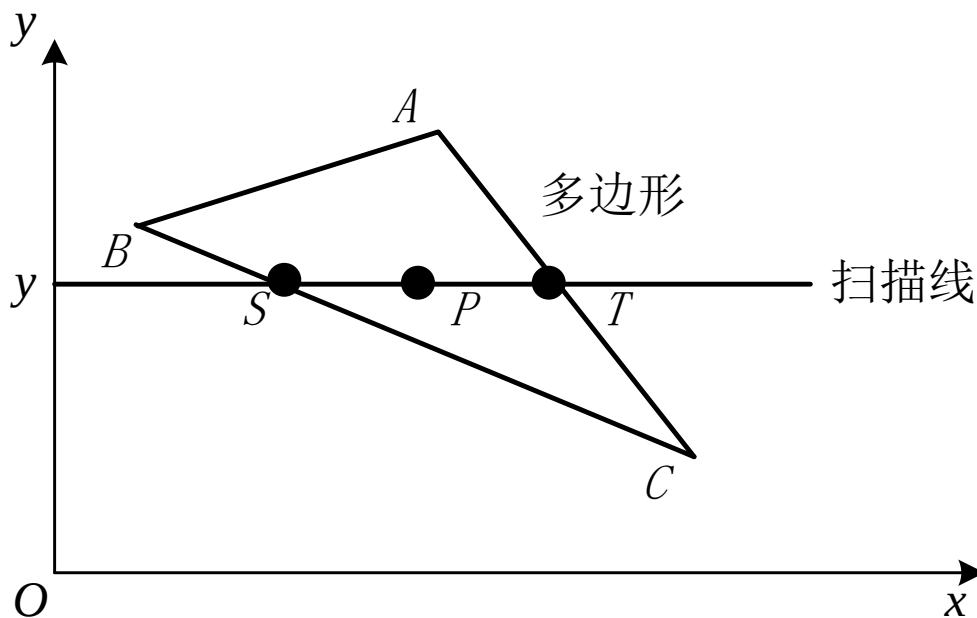


图10.4 Gouraud 明暗处理的双线性插值

Phong明暗处理

- Phong明暗处理方法，又称为**法矢量插值明暗处理**，它对多边形顶点的法矢量进行插值以产生中间各点的**法矢量**，其步骤为：
 - 计算每个多边形顶点处的平均单位法矢量；
 - 用双线性插值方法求得多边形内部各点的法矢量。
 - 最后按光照模型确定多边形内部各点的光强。

□ 矢量双线性插值方法

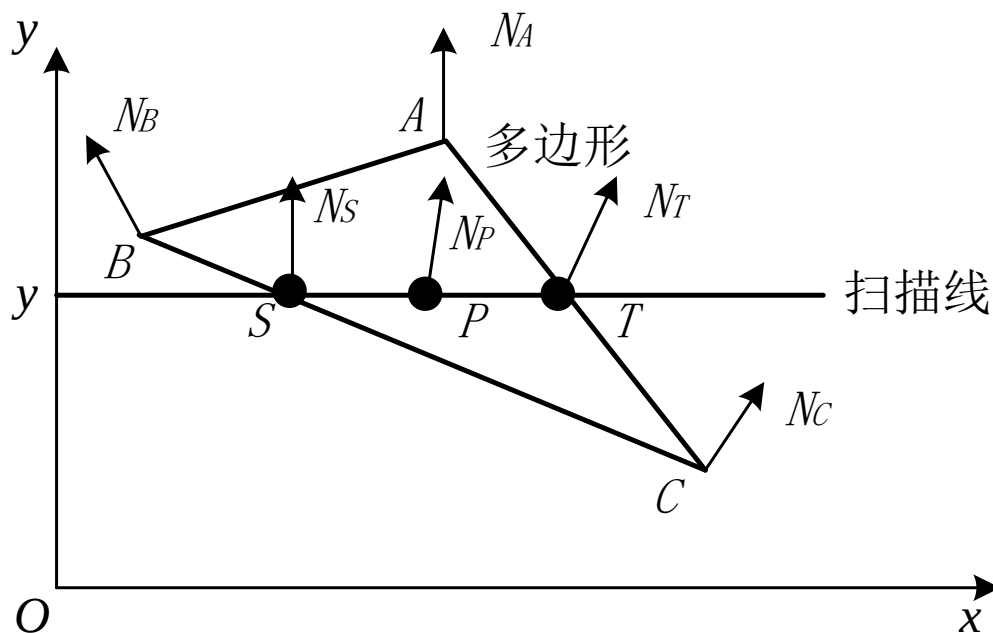
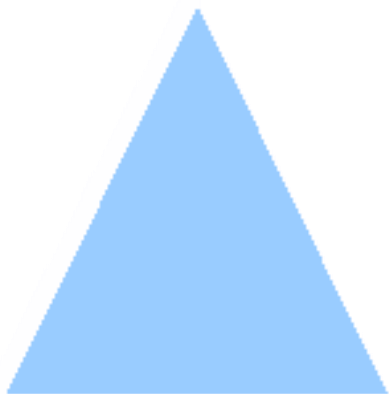
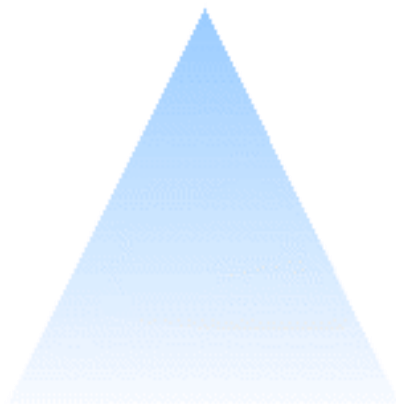


图10.5 Phong明暗处理的矢量双线性插值

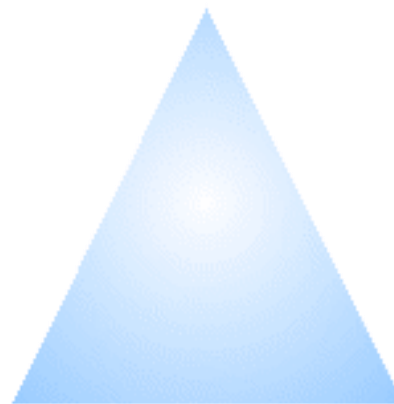
效果比较



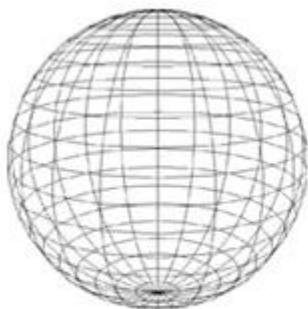
Flat Shading



Gouraud Shading



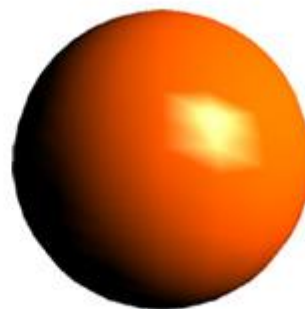
Phong Shading



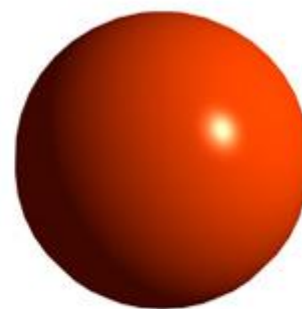
(a)



(b)



(c)



(d)

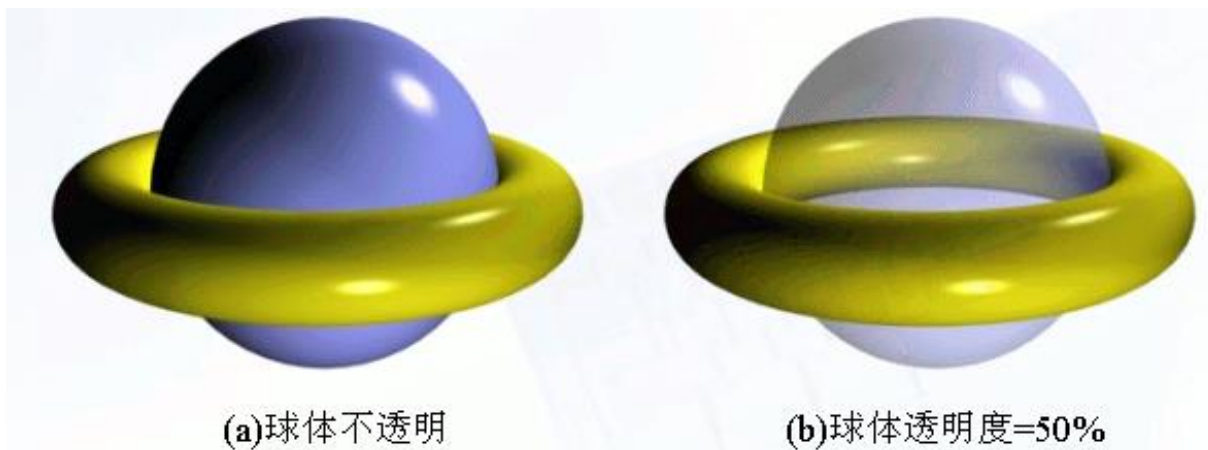
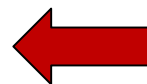
透明处理和阴影

1

透明处理

2

产生阴影



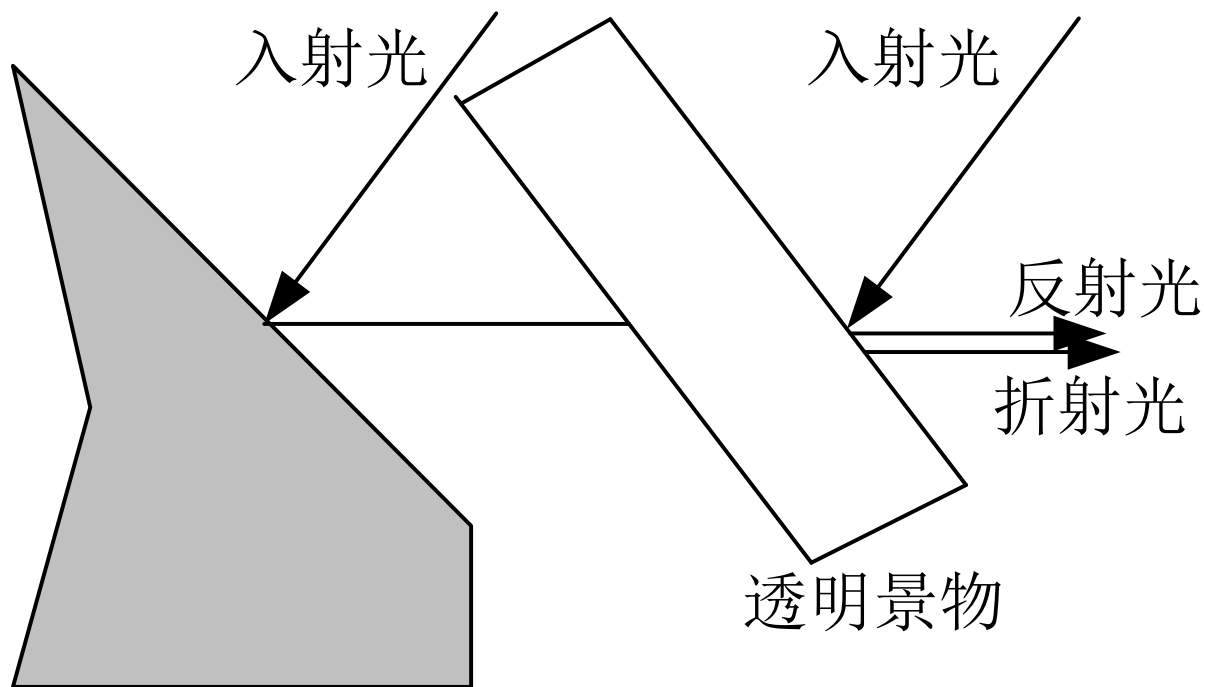


图10.6 透明表面的光强包括反射光和折射光

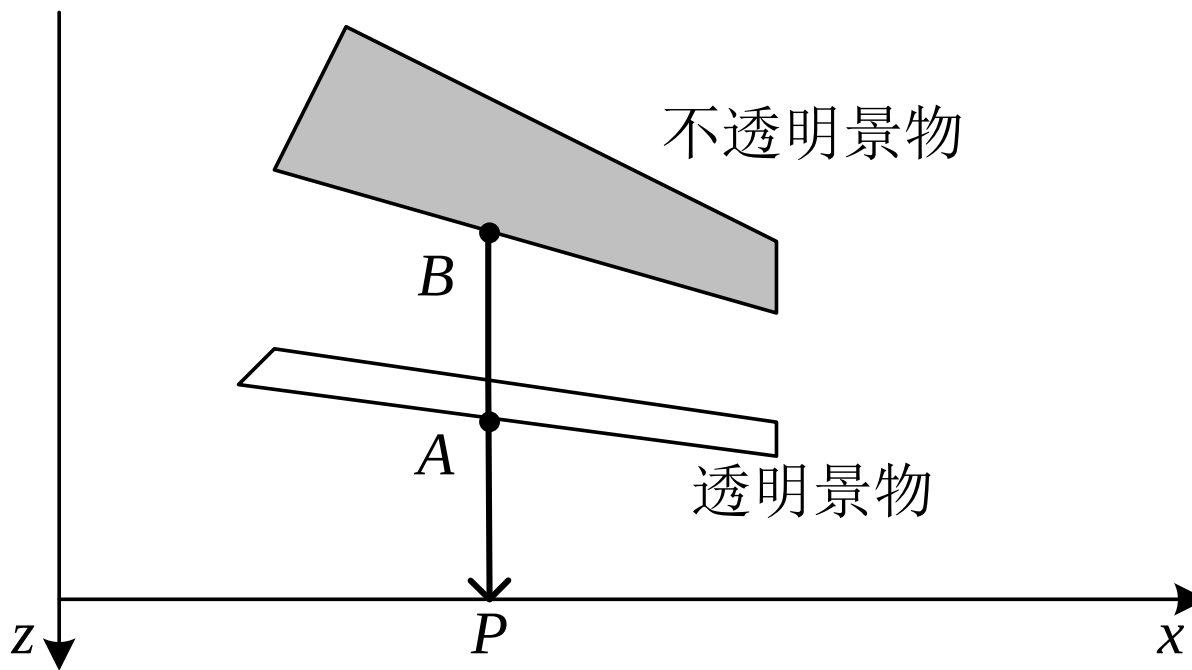


图10.7 简单的透明处理

$$T = \left(\frac{\eta_i}{\eta_r} \sin \theta_i - \cos \theta_r \right) N - \frac{\eta_i}{\eta_r} L$$

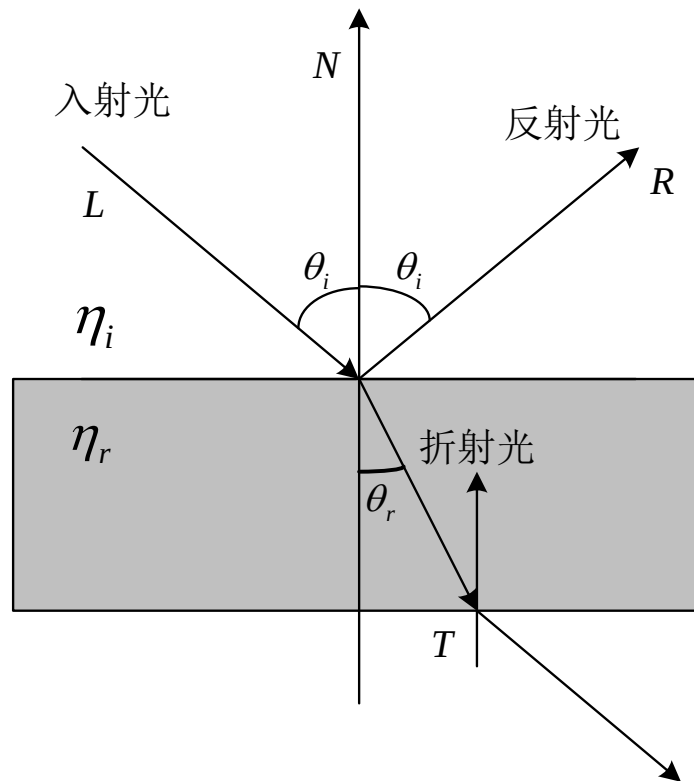


图10.8 光的折射

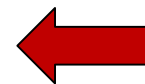
透明处理和阴影

1

透明处理

2

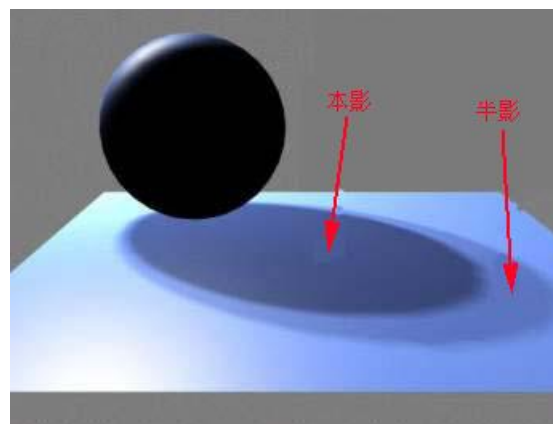
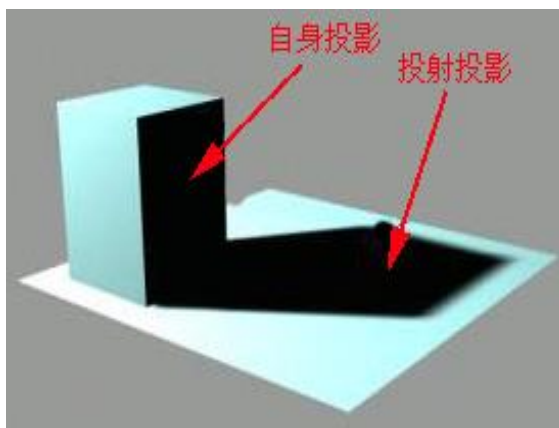
产生阴影



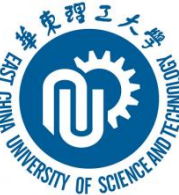
产生阴影

- **阴影**是由于物体截断了光线而产生的，所以如果光源位于物体一侧的话，阴影总是位于物体的另一侧，也就是与光源相反的一侧。
- 从理论上来说，从视点以及从光源看过去都是可见的面不会落在阴影中，只有那些从视点看过去是可见的，而从光源看过去是不可见的面，肯定落在阴影之内。

产生阴影

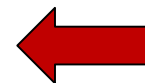


模拟景物表面细节（纹理映射）



1

用多边形模拟表面细节



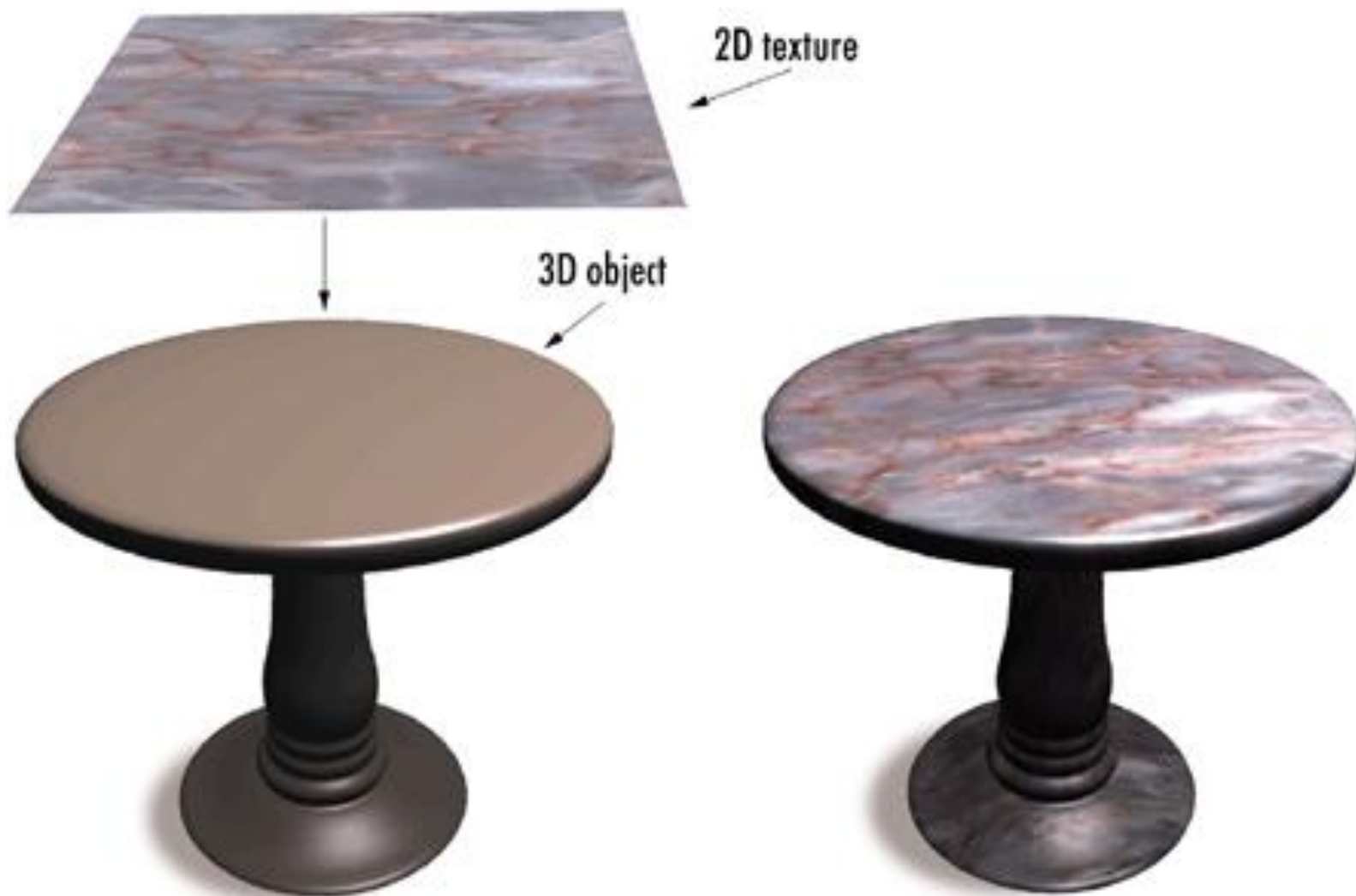
2

纹理的定义和映射

3

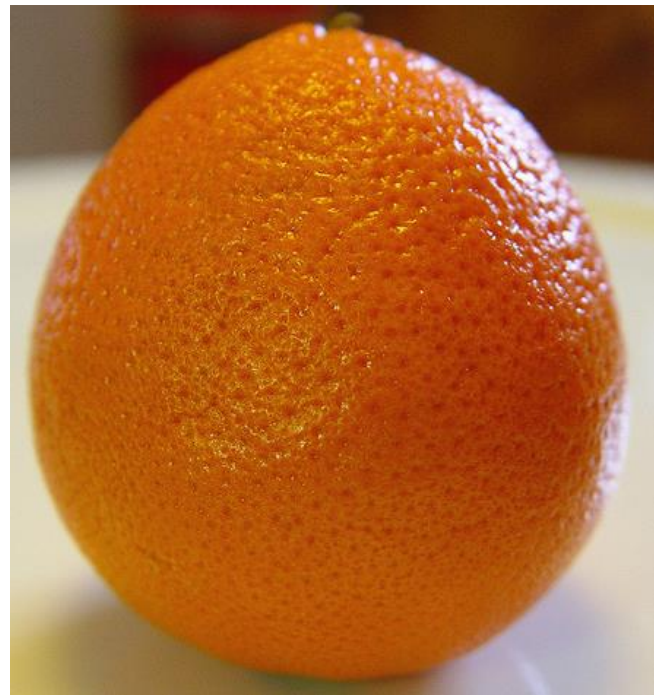
凹凸映射

利用纹理图像增强真实感



复杂物体表面细节表现

- 复杂几何细节：渲染开销巨大



高度复杂细节的几何模型



模拟景物表面细节

- **颜色纹理：通过颜色色彩或明暗度的变化体现出来的表面细节。**
- **几何纹理：由于不规则的细小凹凸造成的。**
- **颜色纹理取决于物体表面的光学属性，而几何纹理由物体表面的微观几何形状决定。**

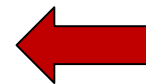
模拟景物表面细节

1

用多边形模拟表面细节

2

纹理的定义和映射

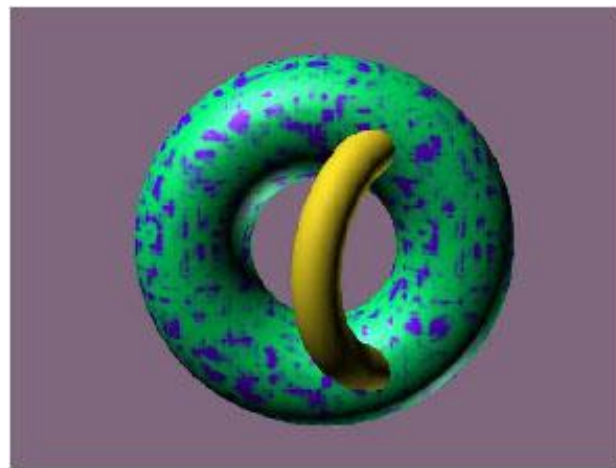
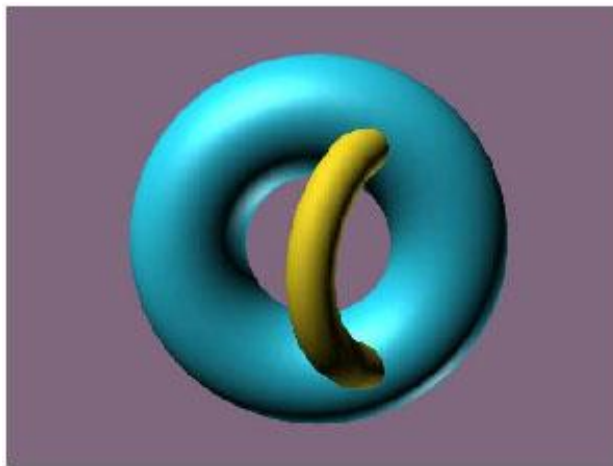


3

凹凸映射

纹理映射和定义

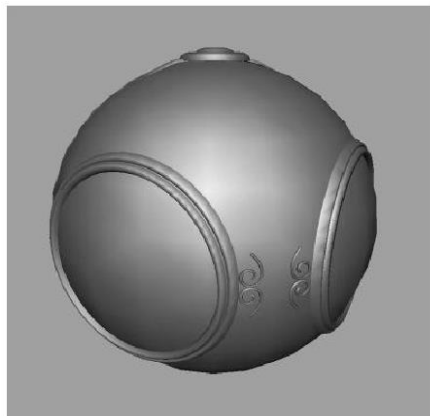
- 生成颜色纹理的一般方法，是预先定义纹理模式，然后建立物体表面的点与纹理模式的点之间的对应。当物体表面的可见点确定之后，以纹理模式的对应点参与光照模型进行计算，就可把纹理模式附到物体表面上。这种方法称为**纹理映射 (Texture Mapping)**。



纹理映射效果图

三种映射方法

- 纹理映射(texture mapping)
- 环境映射（反射映射）(enviornment mapping)
- 凹凸映射 (bump mapping)



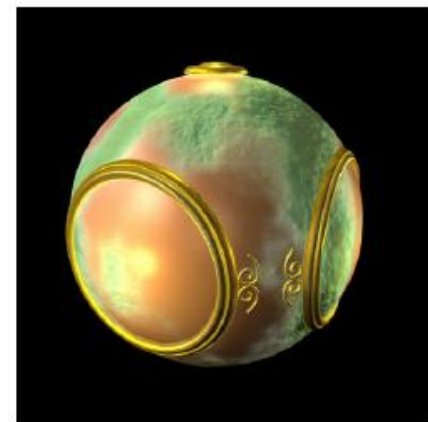
几何模型



纹理映射



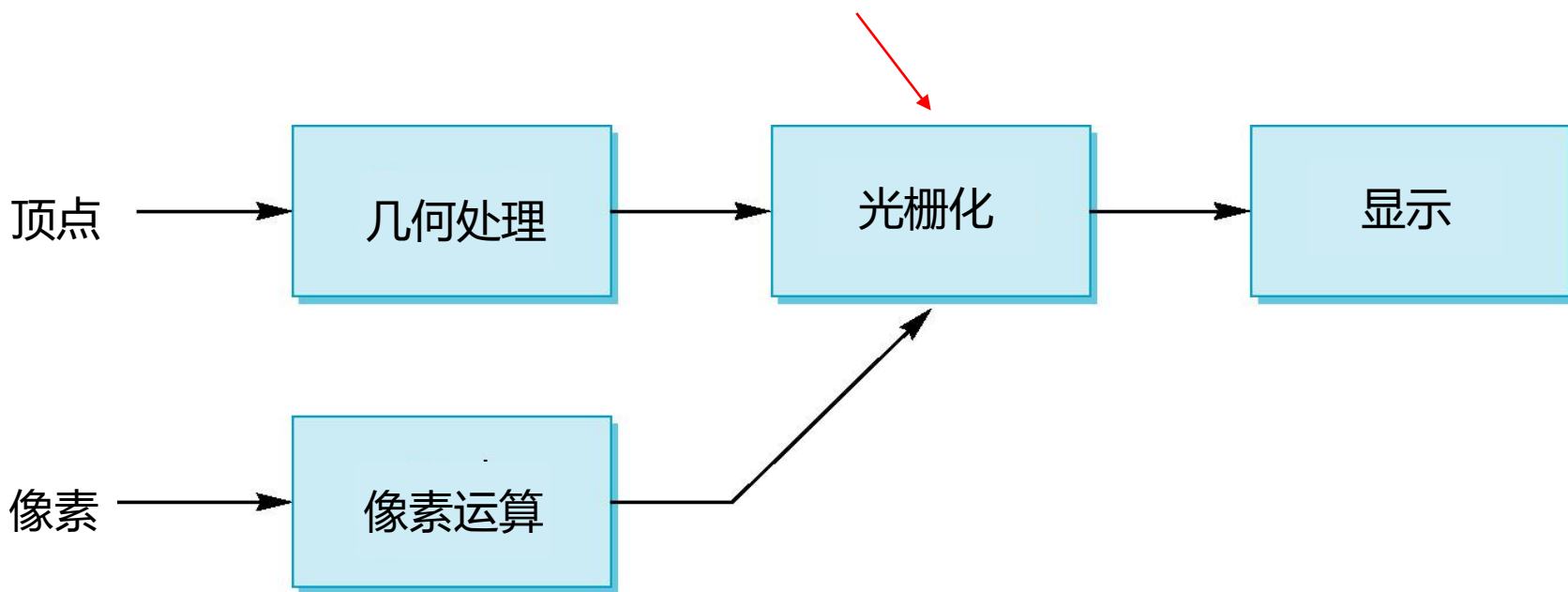
环境映射



凹凸映射

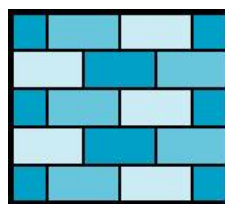
映射在什么地方进行？

- 映射技术是在输出流水线的最后阶段实现的
 - 非常有效，因为在经过所有的操作后，减少了许多不必要的映射

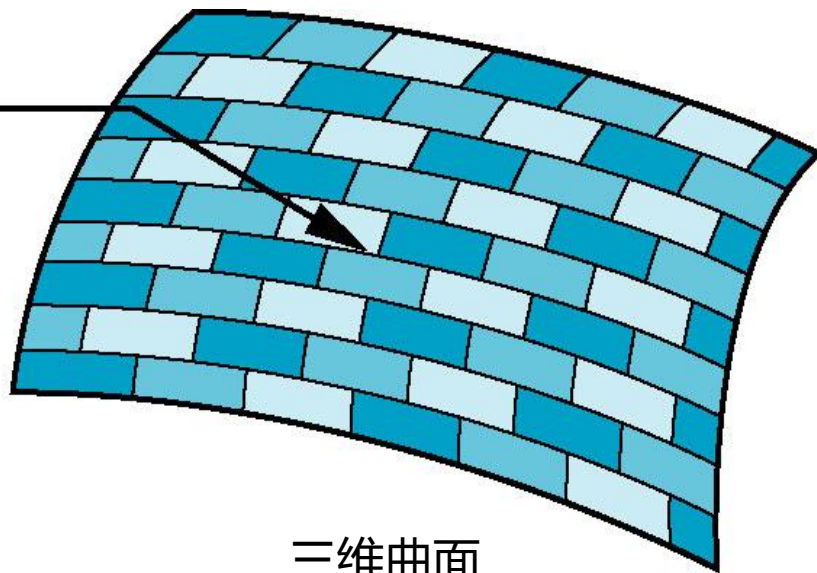


坐标系

- 参数坐标系
 - 可以用来建立曲面
- 纹理坐标系
 - 用来区别要被映射的图像上的点
- 世界坐标系
 - 从概念上说，就是映射发生的地方
- 屏幕坐标系
 - 最终图像生成的地方

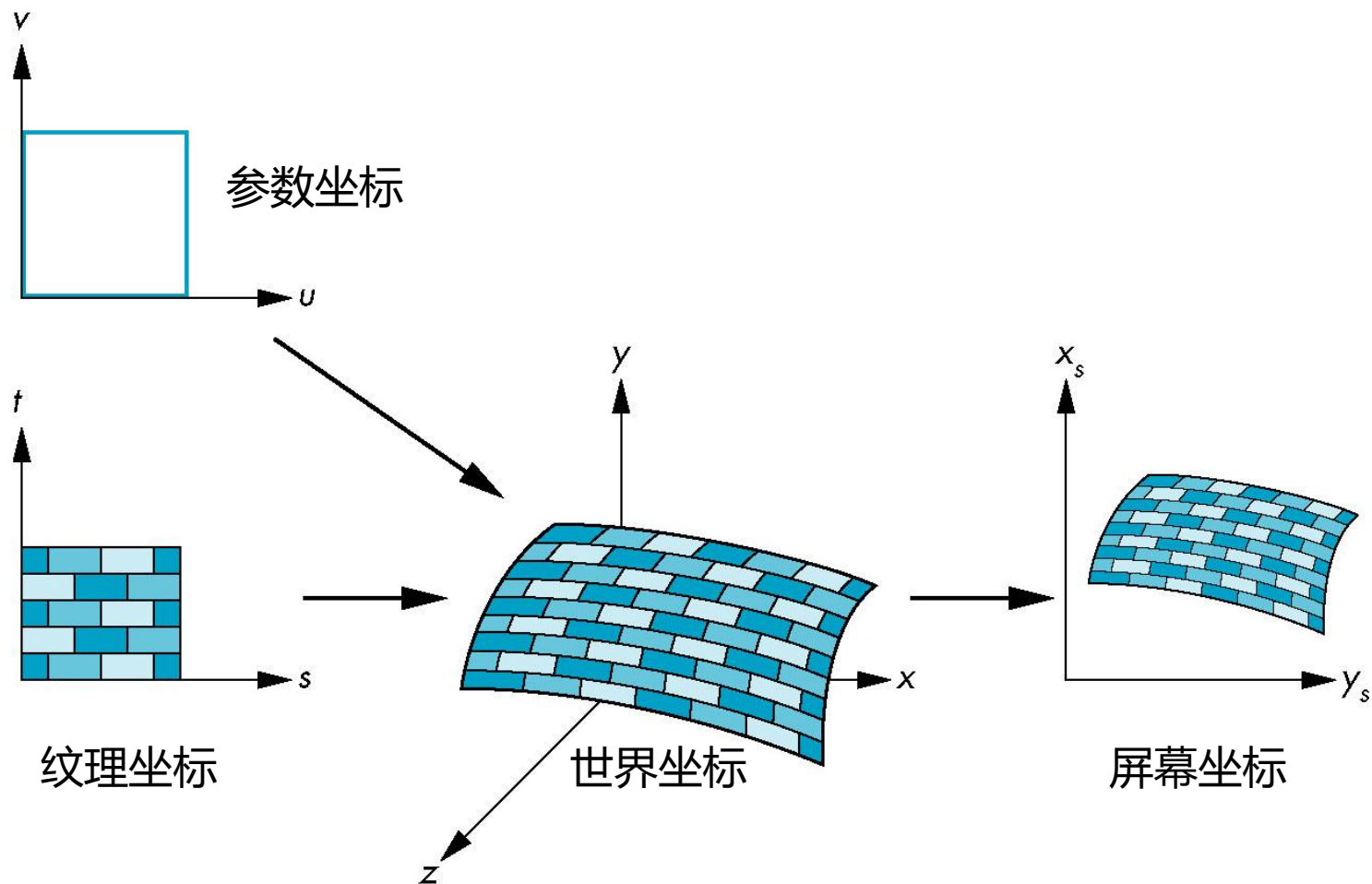


二维图像



三维曲面

纹理映射框架

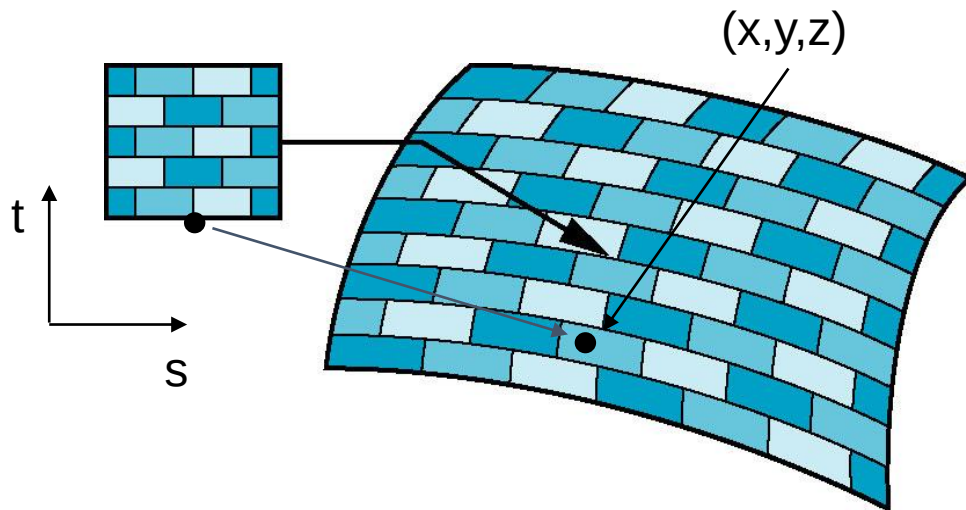


映射函数

- 基本问题就是如何定义映射
- 考虑从纹理坐标到曲面上一点的映射
- 直观地看，应当需要三个函数

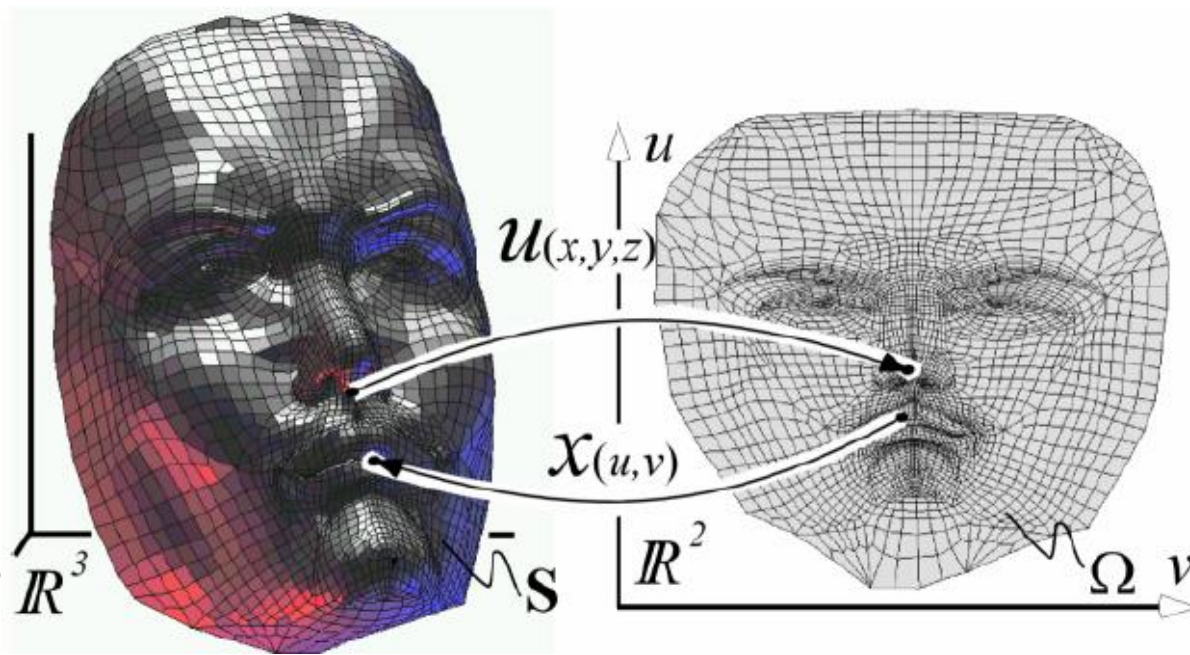
$$x = x(s,t), y = y(s,t), z = z(s,t)$$

- 虽然从概念上讲，最终必定用到上述的函数，但实际采用的是却是间接的方法



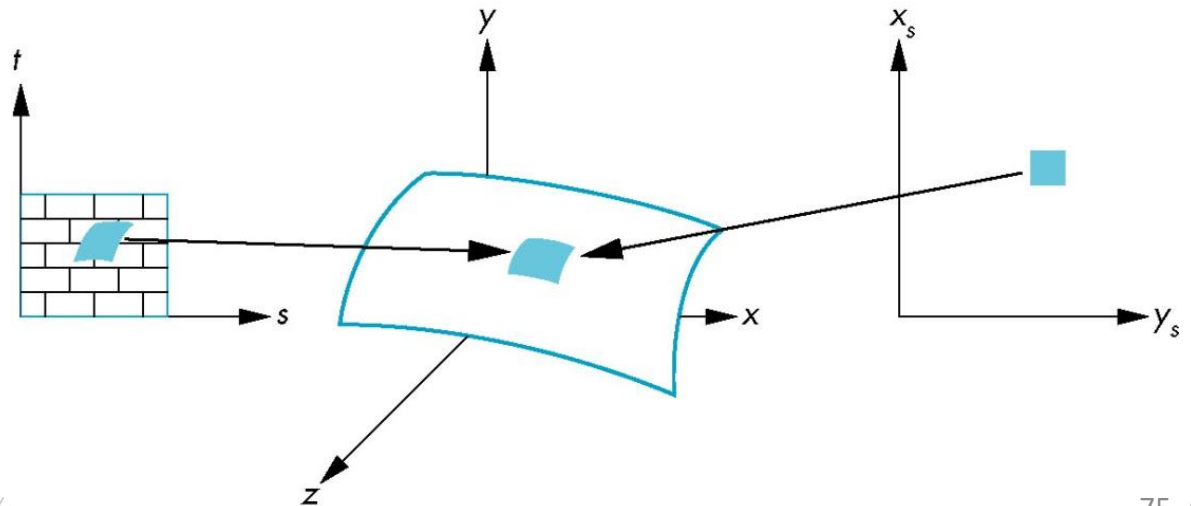
逆映射

- 我们需要的是逆向操作
 - 给定一个像素，我们想知道它对应于对象上的哪个点
 - 给定对象上的一个点，我们想知道它对应于纹理中的哪个点
- 此时需要如下形式的映射
 - $s = s(x,y,z), t=t(x,y,z)$
- 这样的函数一般是很难求出来的？
 - 参数化！



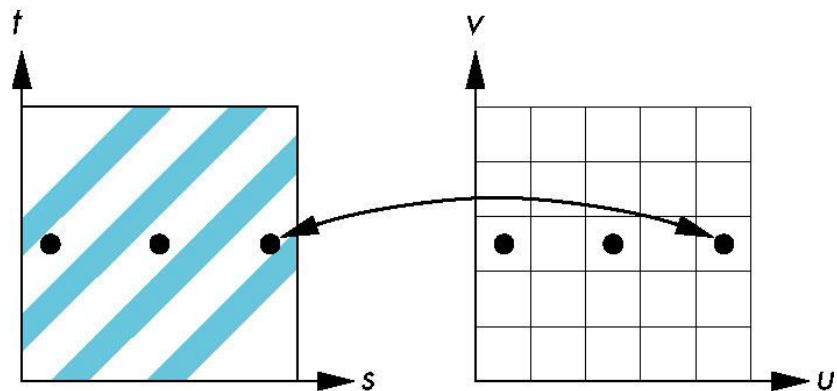
实际困难

- 假设要计算中心在 (x_s, y_s) 的一个矩形像素的颜色，中心点对应于对象上的点 (x, y, z)
- 如果对象是弯曲的，那么矩形像素的原像是一个曲边四边形
- 这个曲边四边形在纹理上的原像才是对当前矩形像素的颜色有贡献的纹理元素



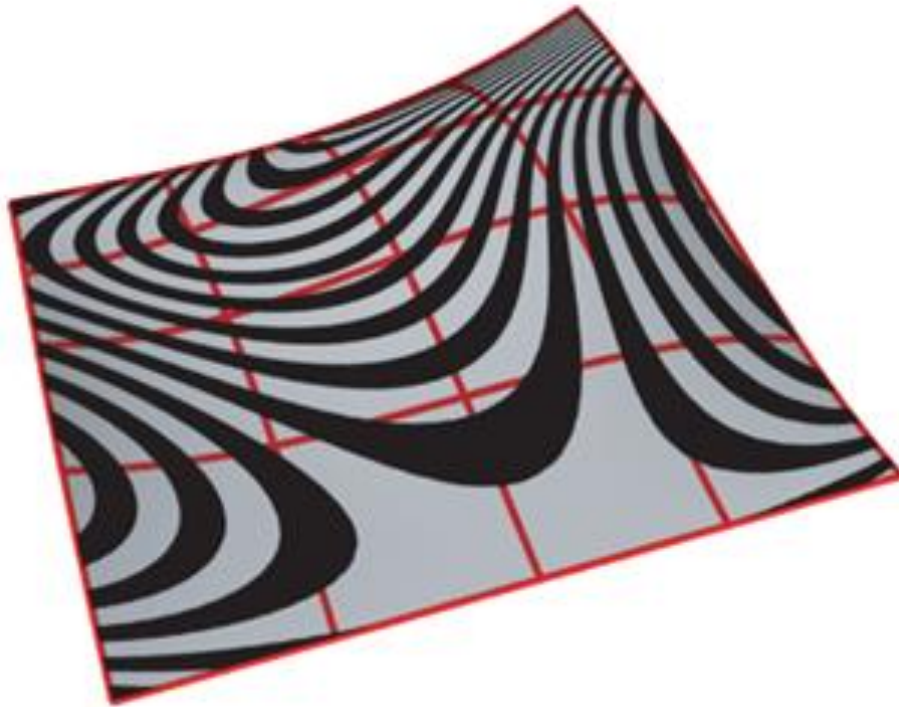
解决方法

- 先不管如何定义逆映射，只考虑如何确定当前像素的颜色（或者说明暗效果）
- 一种方法是用当前像素块的原像的中心对应的纹理元素的颜色
 - 走样（aliasing）



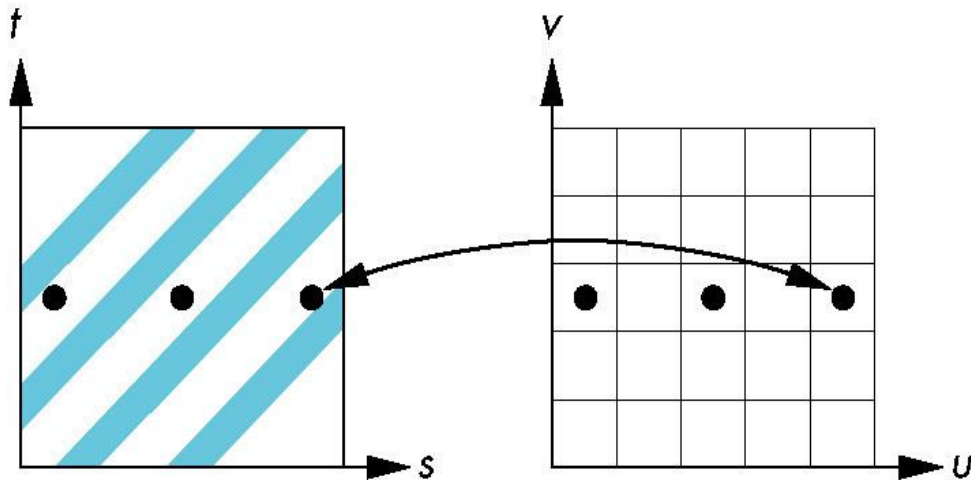
Moiré效果

- 当不考虑像素是有一定的大小时，在图像中会导致moiré效果，即出现波纹



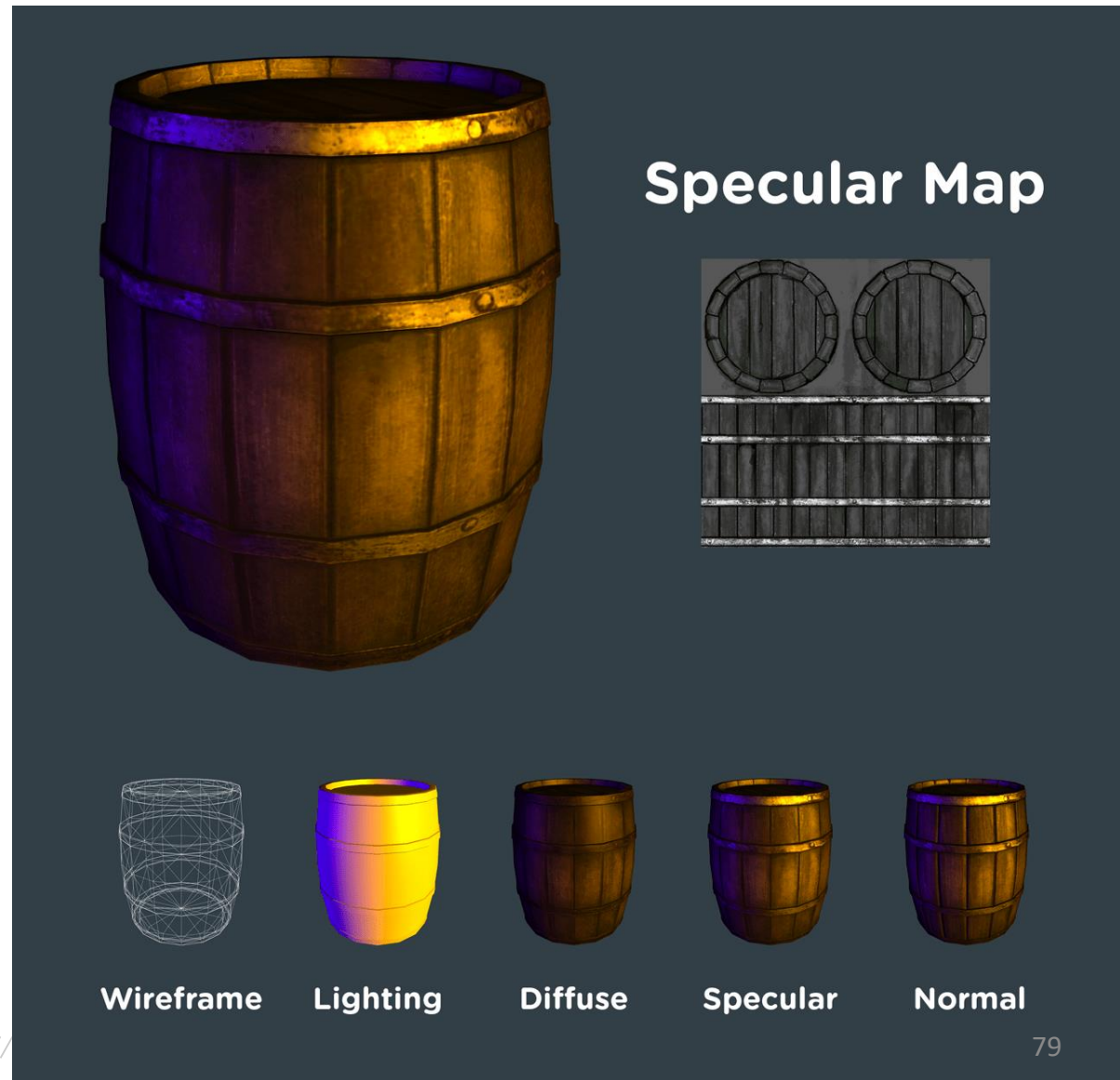
“更好”的方法

- 用在纹理原像上对应区域的明暗效果的平均值赋给当前像素
 - 很难实现
 - 也不是完美的
 - 右图经上述处理后也得不到所需要的结果
 - 但这个问题来自于低分辨率的局限
 - 对于规则纹理，这种效果非常明显



Texture can store many...

- Ambient map
- Diffuse map
- Specular map
- ...



曲面映射的确定

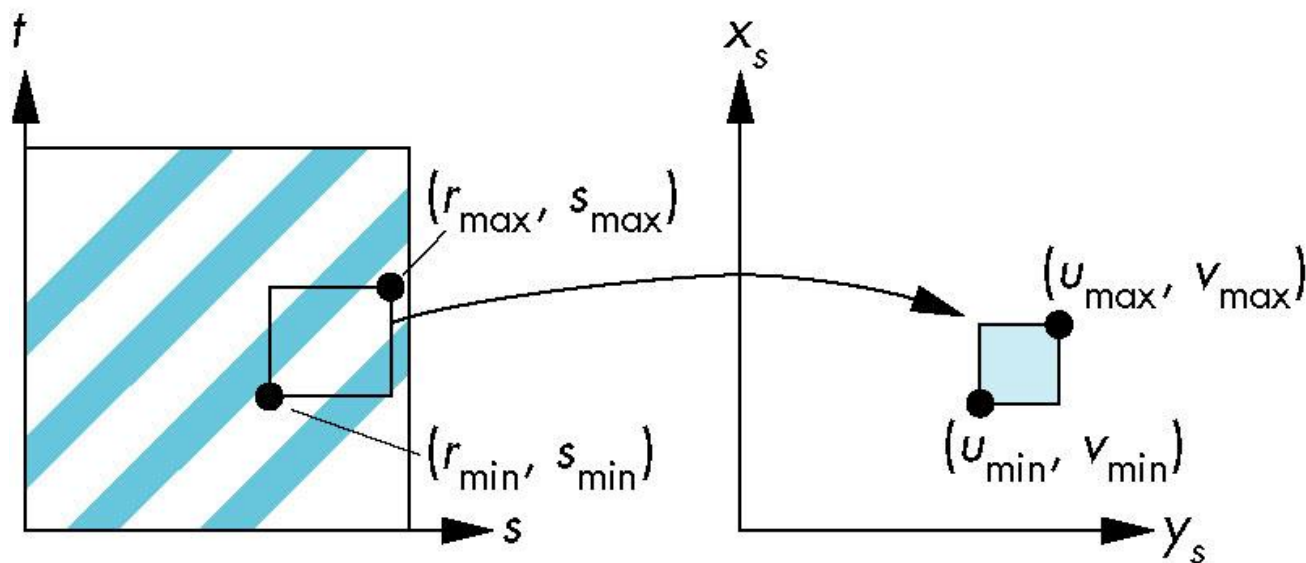
- 考虑由参数方程定义的曲面

$$p(u,v)=(x(u,v),y(u,v),z(u,v))$$

- 此时通常采用如下形式从纹理元素对应到曲面上的点

$$u = as + bt + c, v = ds + et + f$$

- 只要 $ae \neq bd$, 上述映射是可逆的

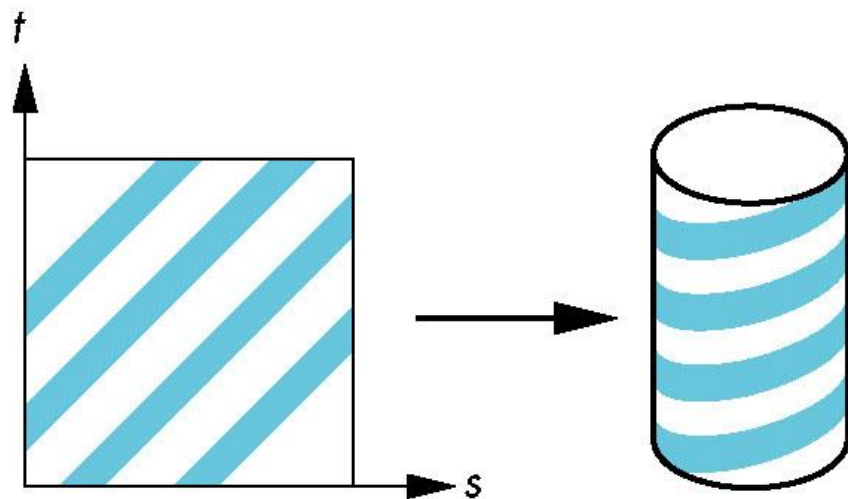


$$u = u_{\min} + \frac{s - s_{\min}}{s_{\max} - s_{\min}} (u_{\max} - u_{\min})$$

$$v = v_{\min} + \frac{t - t_{\min}}{t_{\max} - t_{\min}} (v_{\max} - v_{\min})$$

两步映射

- 解决映射问题的另外一种方法
- 首先把纹理映射到一个简单的中间曲面上
- 例如：映射到圆柱上



圆柱映射

- 假设纹理坐标在单位正方形 $[0,1]^2$ 内变化, 圆柱高 h , 半径 r
- 那么圆柱的参数方程为
$$x = r \cos(2\pi s), y = r \sin(2\pi s), z = ht$$
- 从纹理坐标到圆柱面上没有变形
- 适合于构造与无底的圆柱面拓扑同构的曲面上的纹理

球映射

- 球的参数方程

$$x = r \cos(2\pi s), y = r \sin(2\pi s) \cos(2\pi t), z = r \sin(2\pi s) \sin(2\pi t)$$

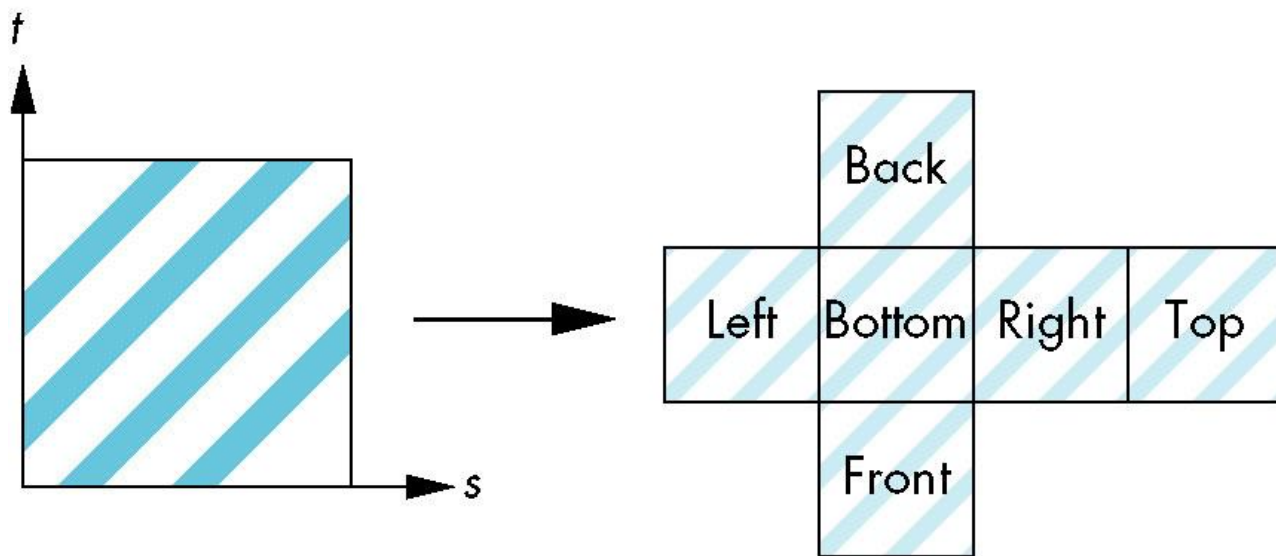
- 类似于地图绘制中的映射

- 肯定有变形

- 用在环境映射中

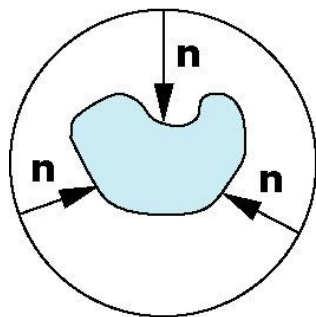
立方体映射

- 适合应用于正交投影
- 也用在环境映射中

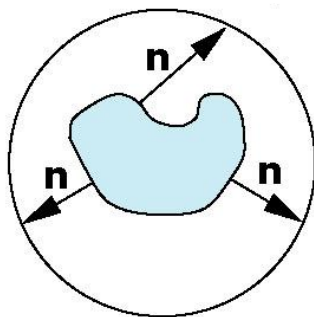


两步映射中的第二个映射

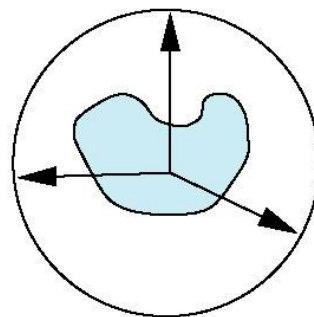
- 从中间对象到实际对象的映射
 - 从中间形状的法向到实际对象
 - 从实际对象的法向到中间形状
 - 从中间形状的中心开始的向量



(a)

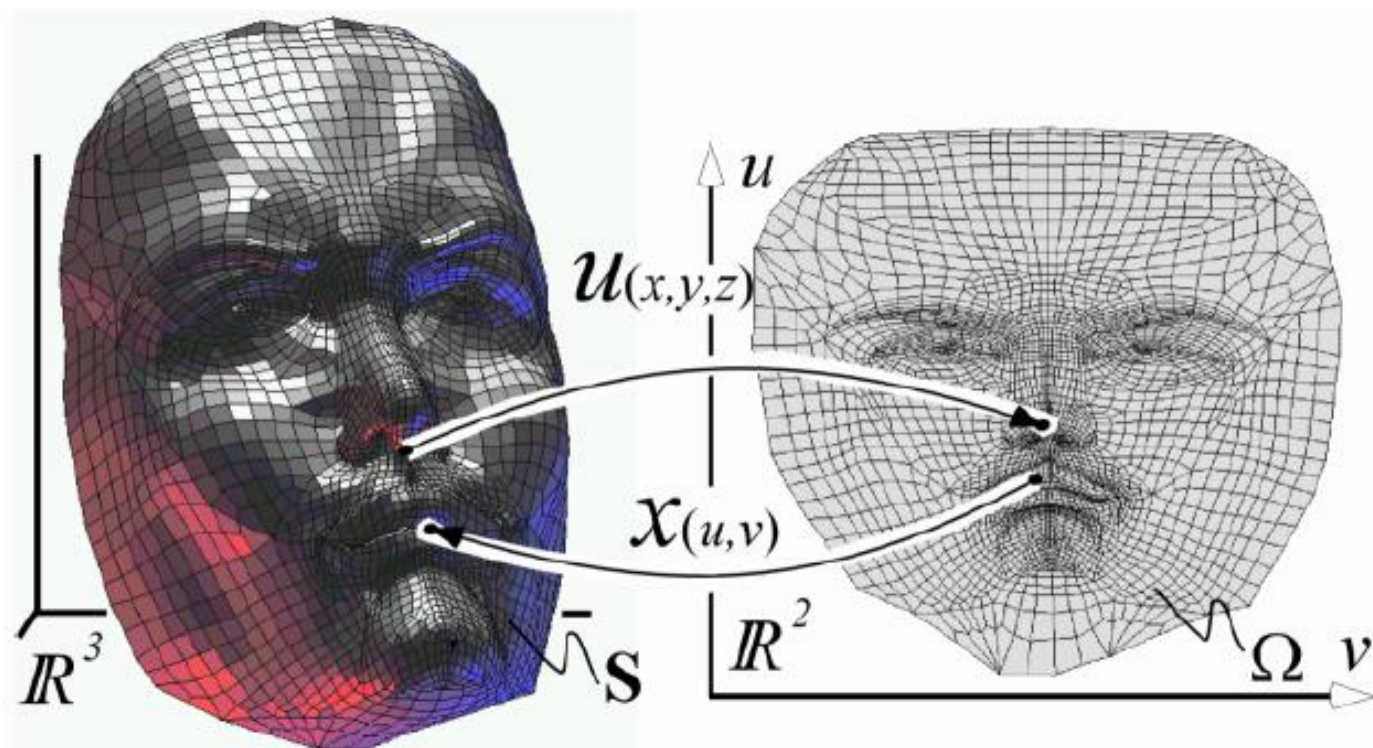


(b)

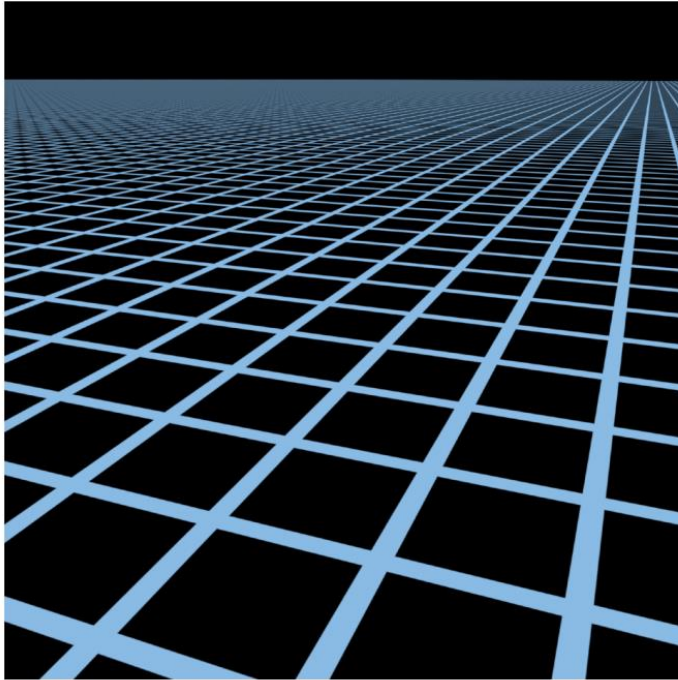


(c)

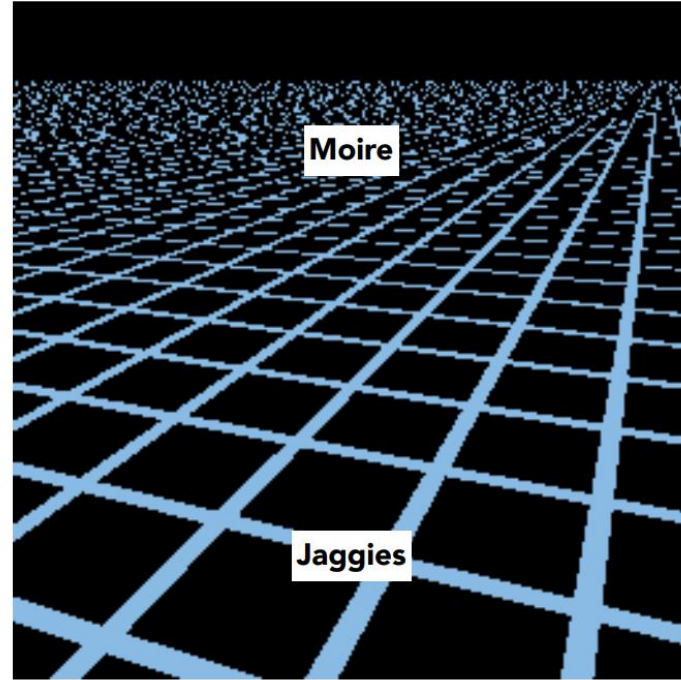
参数化方法



MipMap



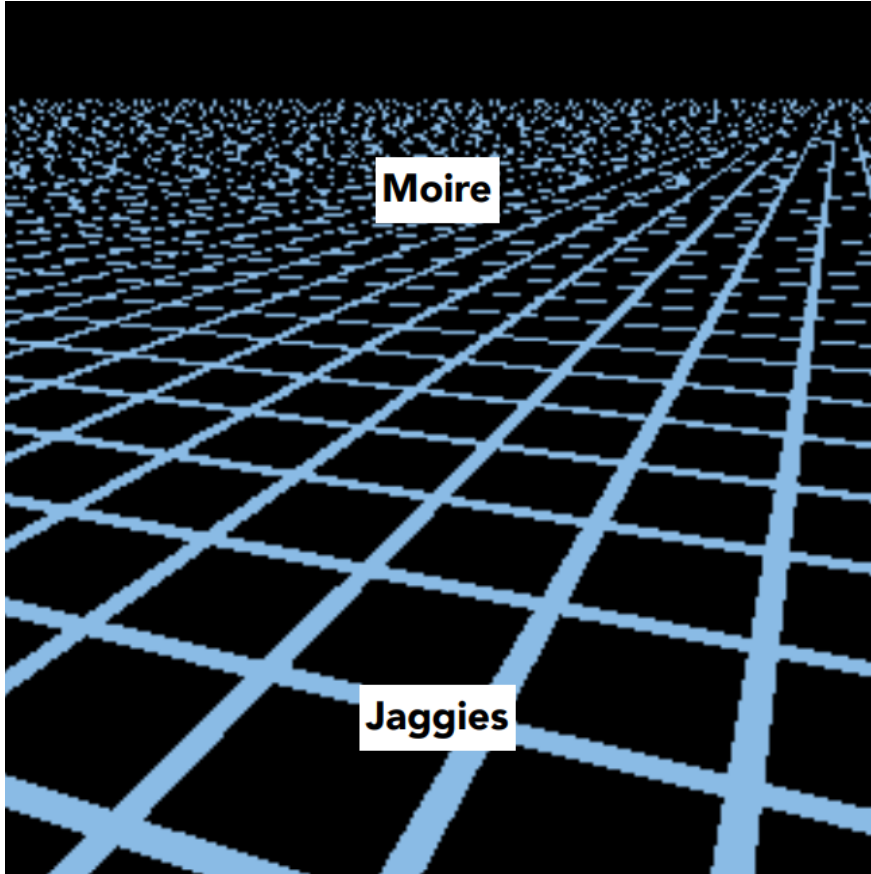
Reference



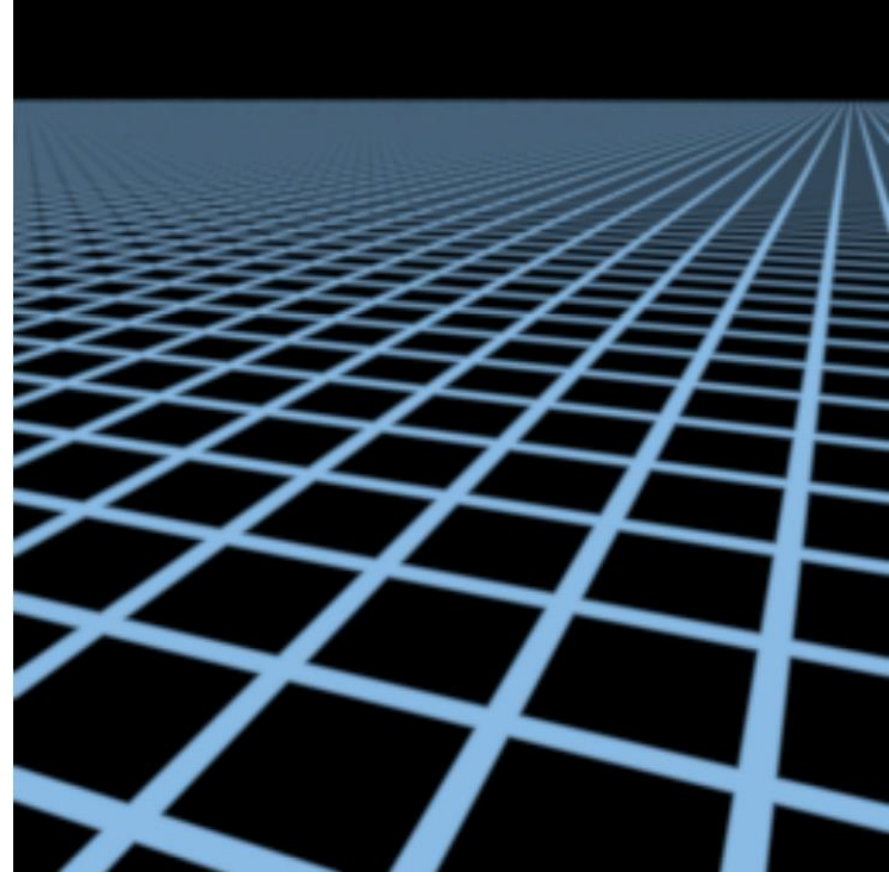
Point sampled

- MipMap: 把一张贴图按照2的倍数进行缩小。直到1X1。把缩小的图都存储起来。在渲染时，根据一个像素离眼睛为之的距离，来判断从一个合适的图层中取出texel（纹理）颜色赋值给像素。

MipMap

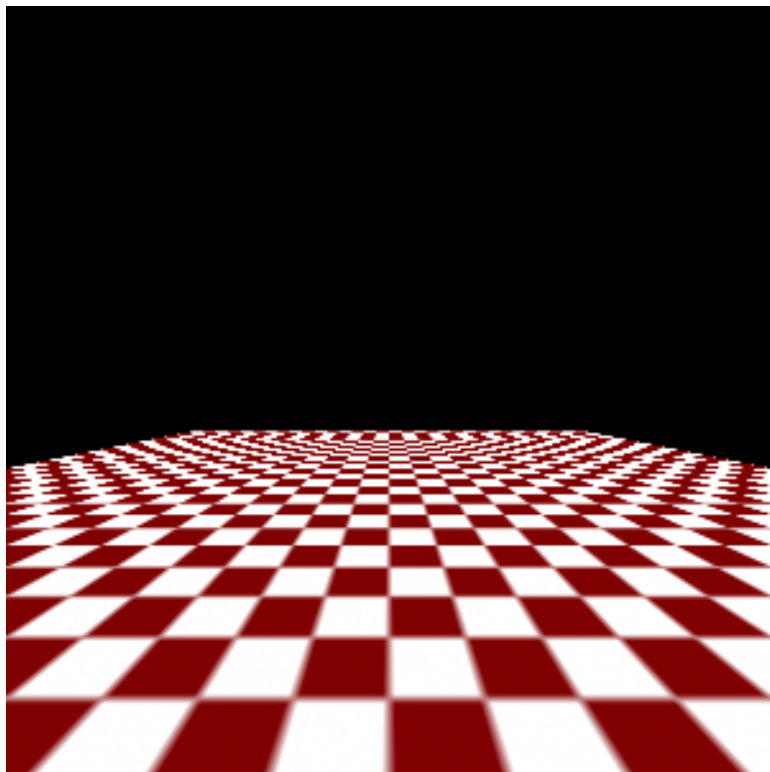


无mipmap

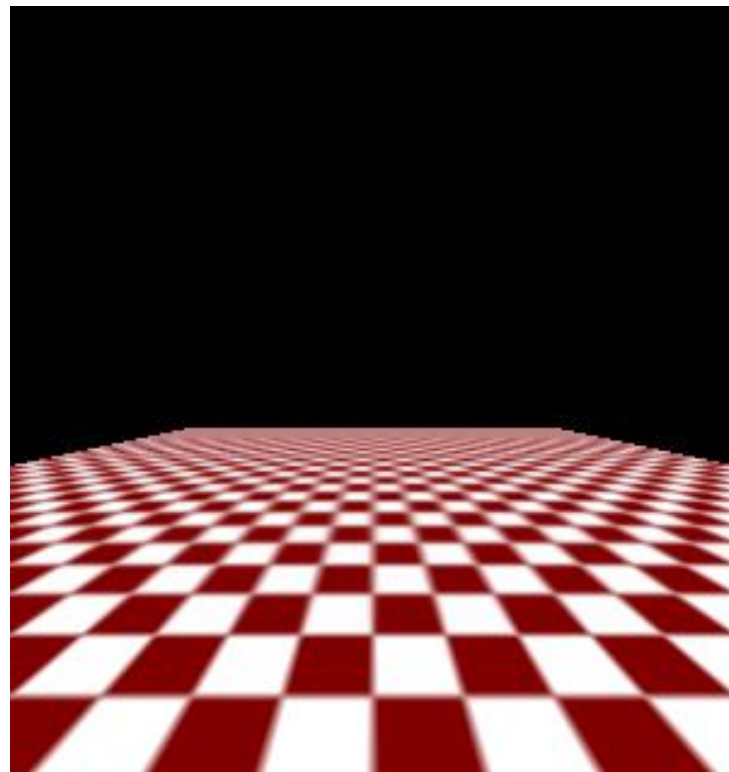


有mipmap

有无mipmap的对比

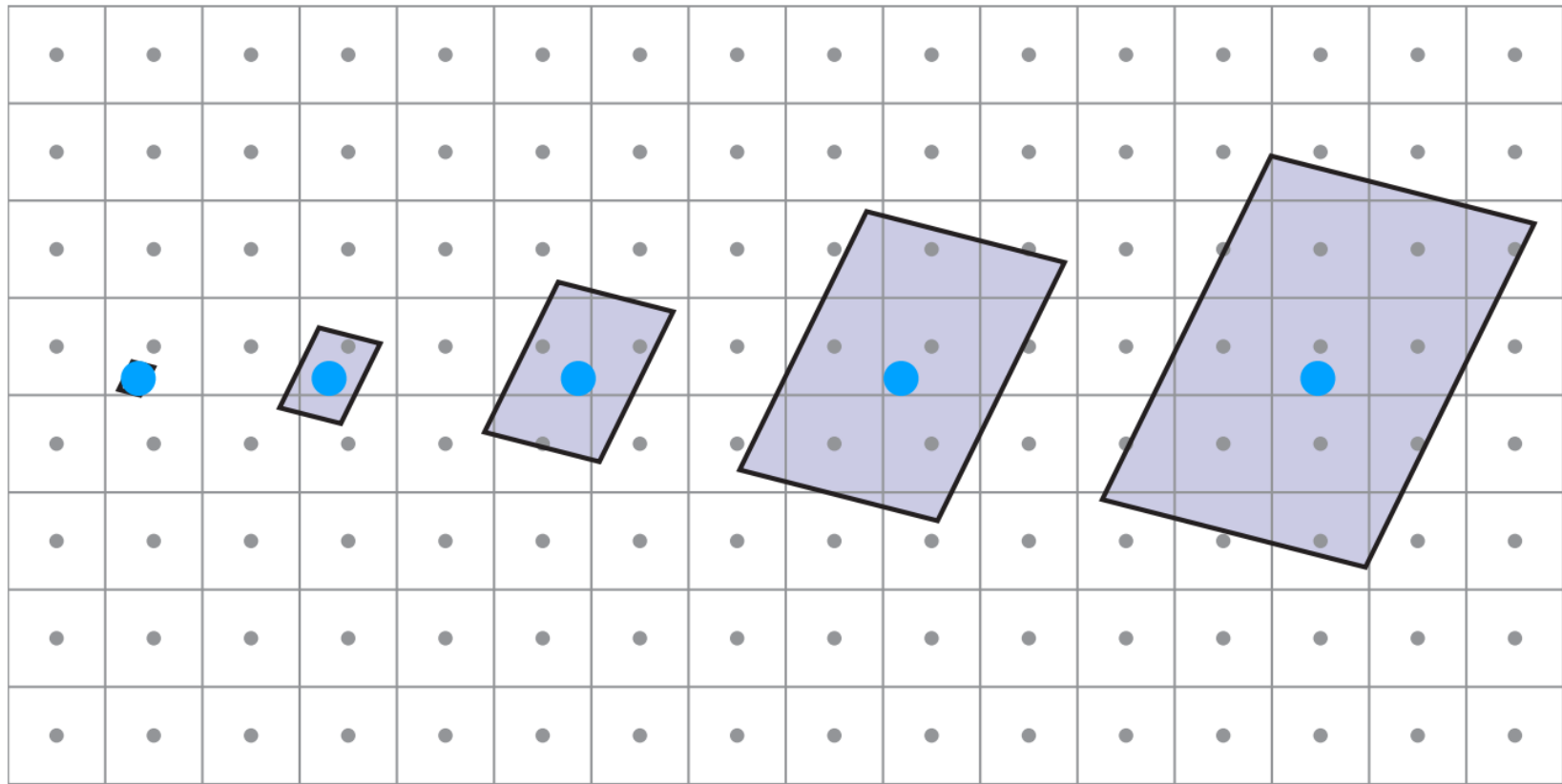


无mipmap



有mipmap

原因：采样不足的问题



Upsampling
(Magnification)



Downsampling
(Minification)

Mipmap的提出 (Williams 1983)



Level 0 = 128x128



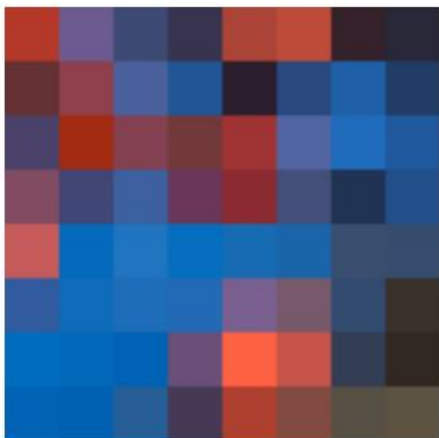
Level 1 = 64x64



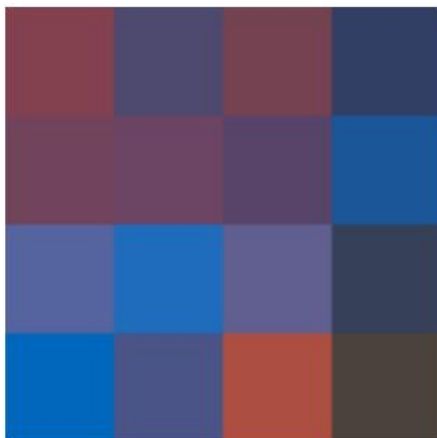
Level 2 = 32x32



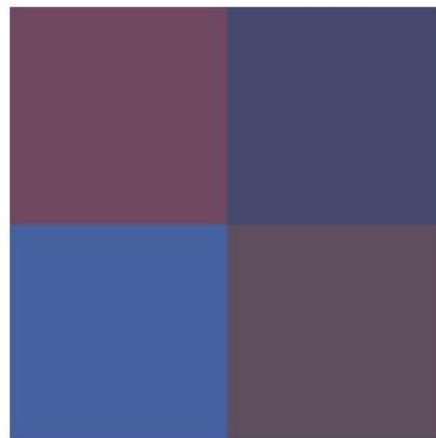
Level 3 = 16x16



Level 4 = 8x8



Level 5 = 4x4

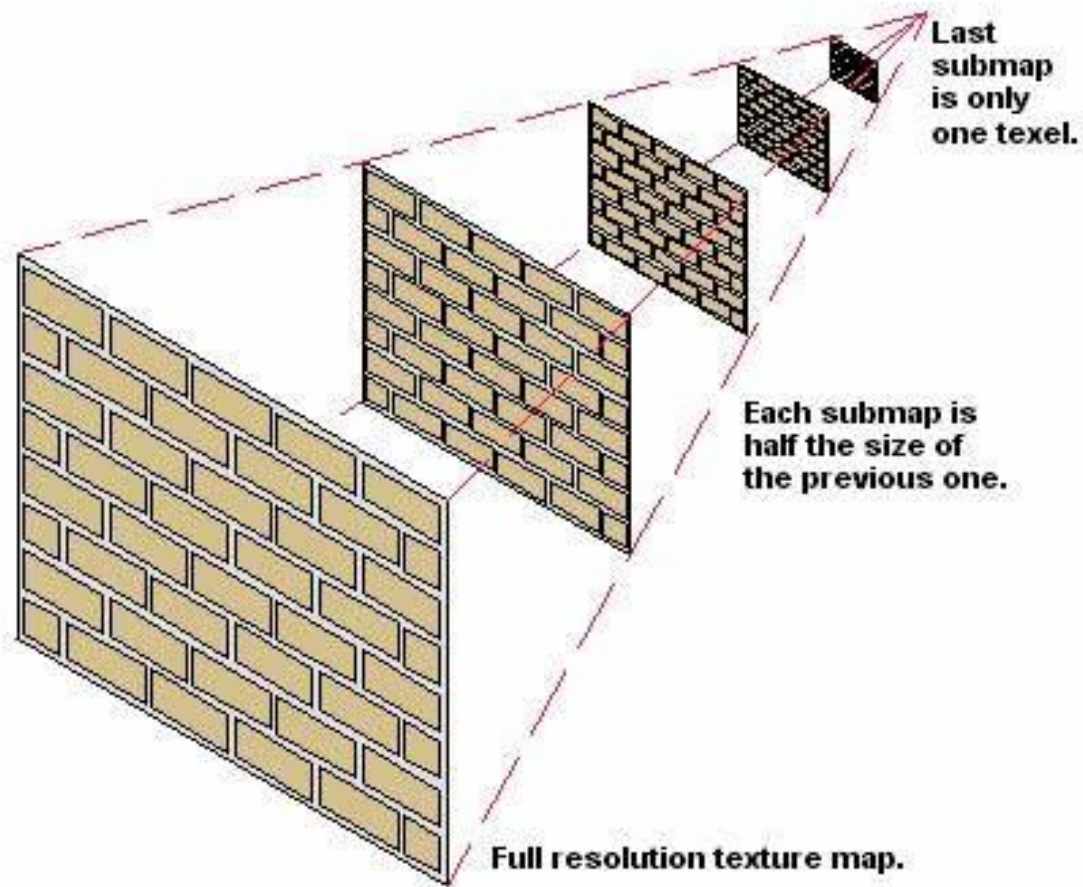


Level 6 = 2x2

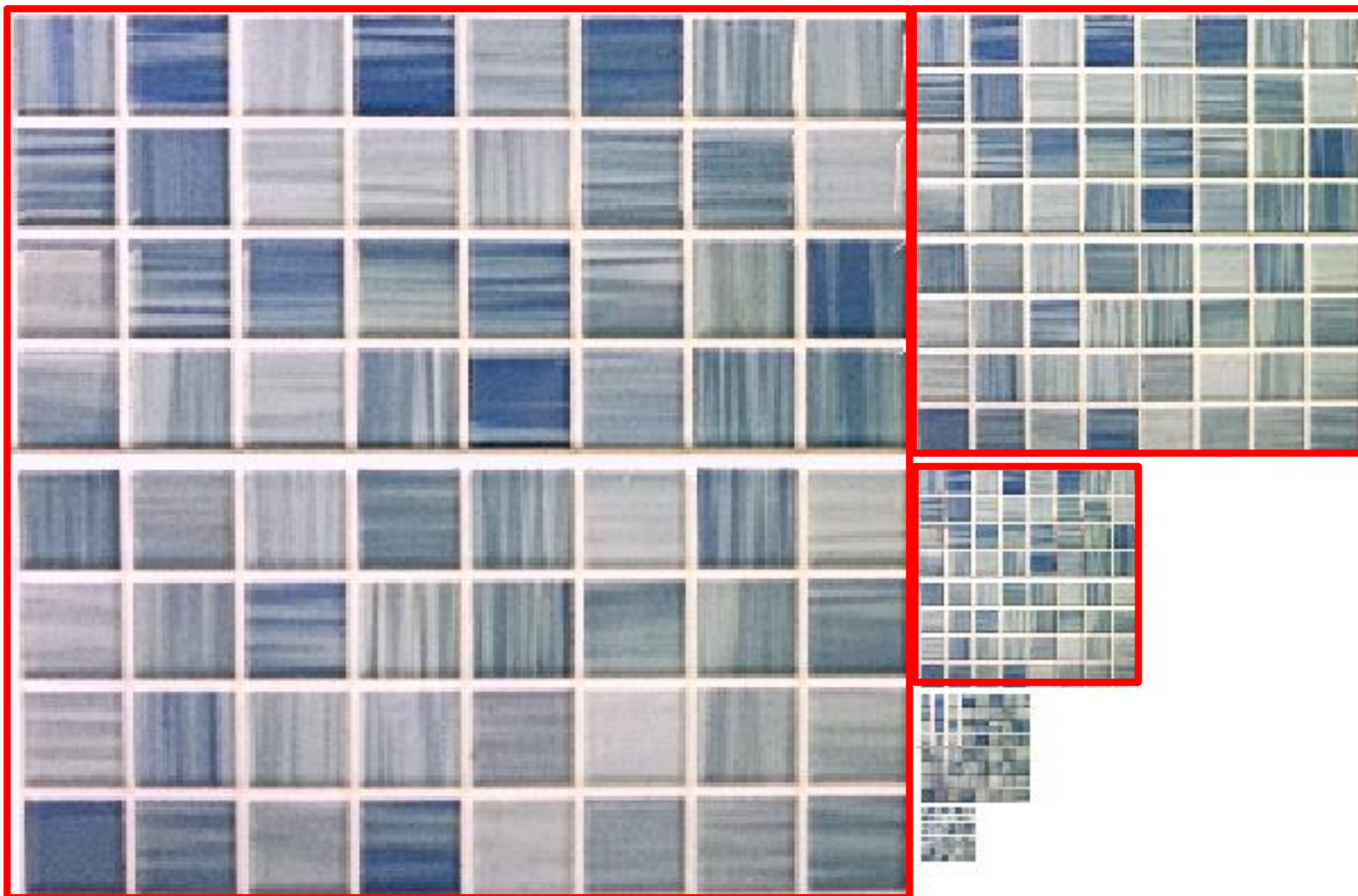


Level 7 = 1x1

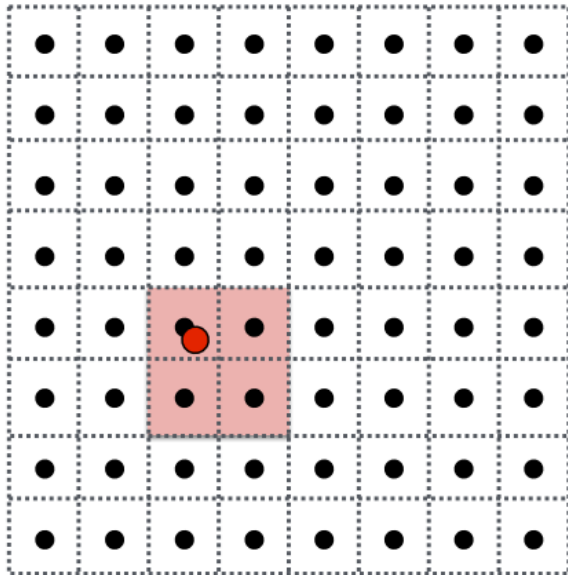
Image Pyramid



Mipmap存储

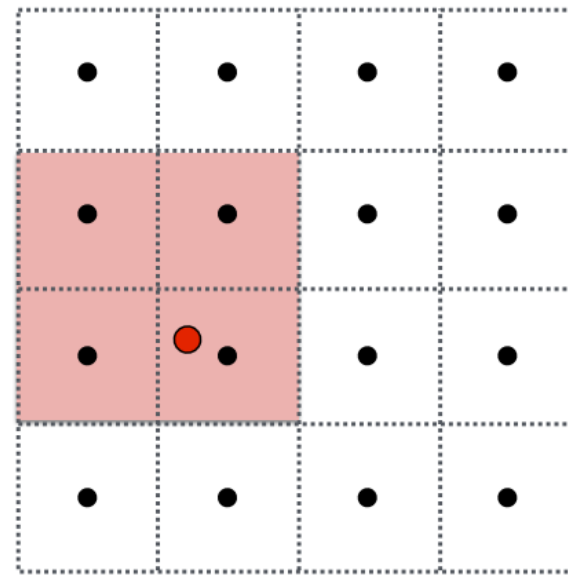


Mipmap blending



Mipmap Level D

Bilinear result



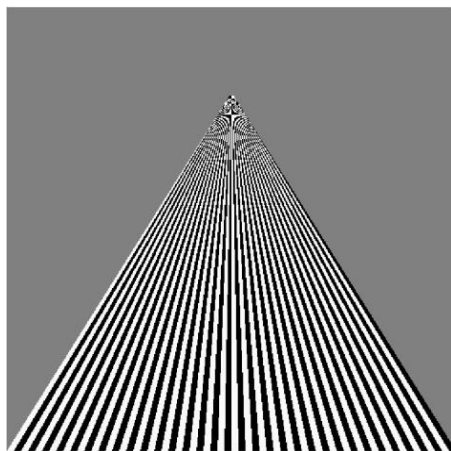
Mipmap Level D+1

Bilinear result

Linear interpolation based on continuous D value

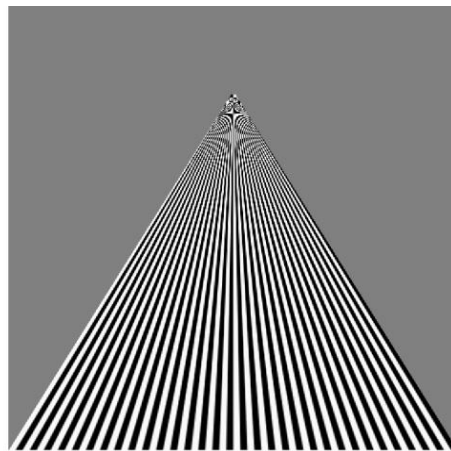
示例

点取样



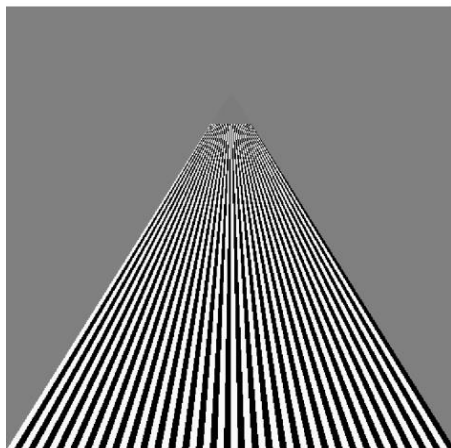
(a)

线性滤波



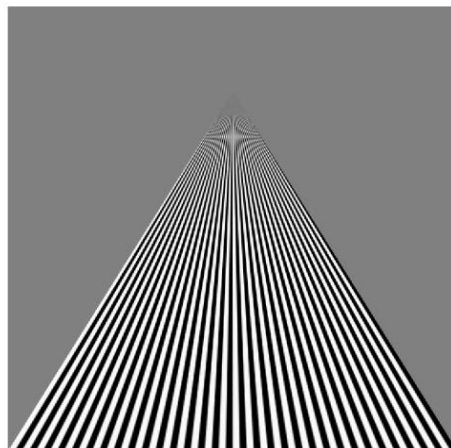
(b)

mipmapped
后的点取样



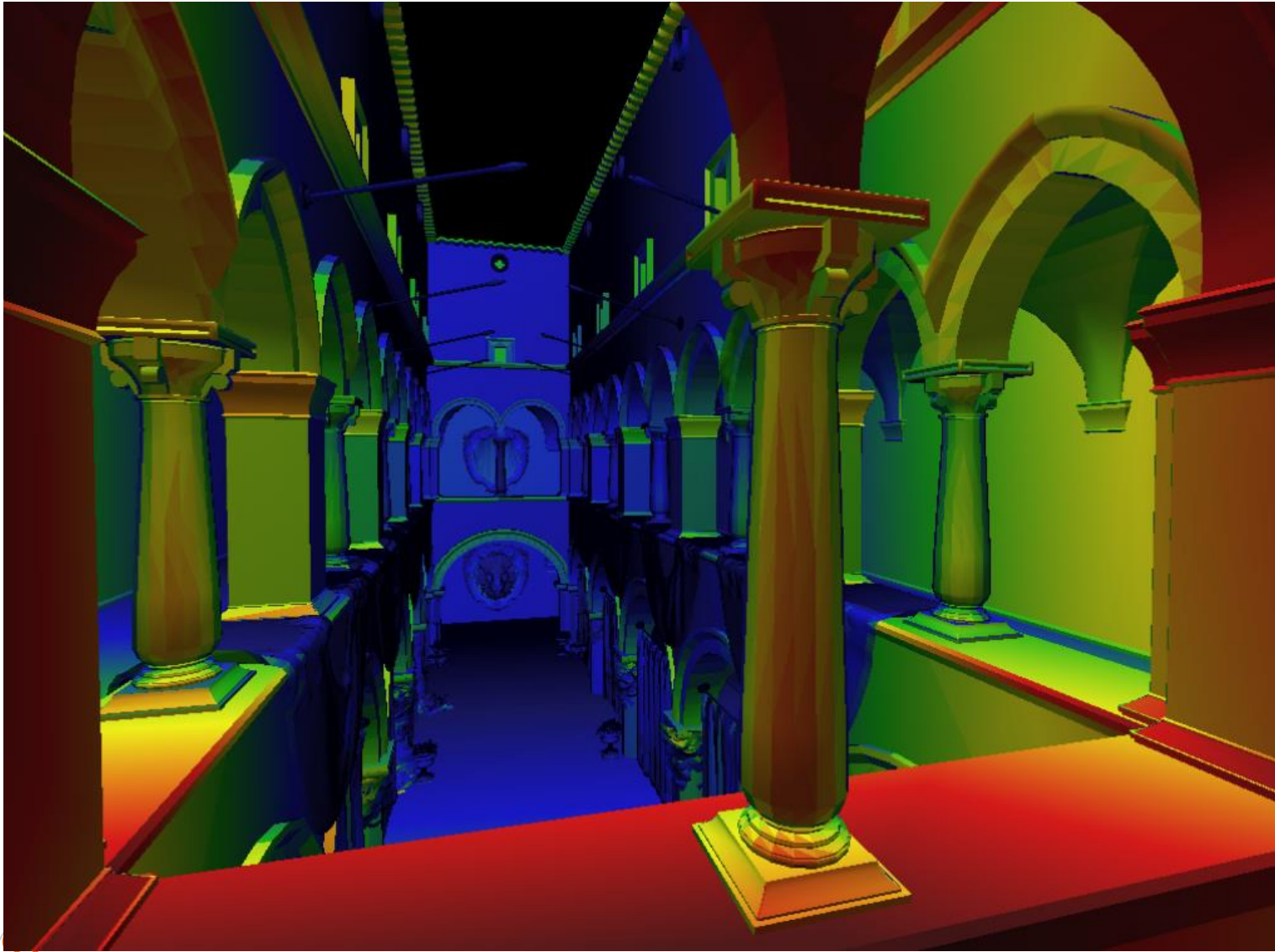
(c)

mipmapped
的线性滤波

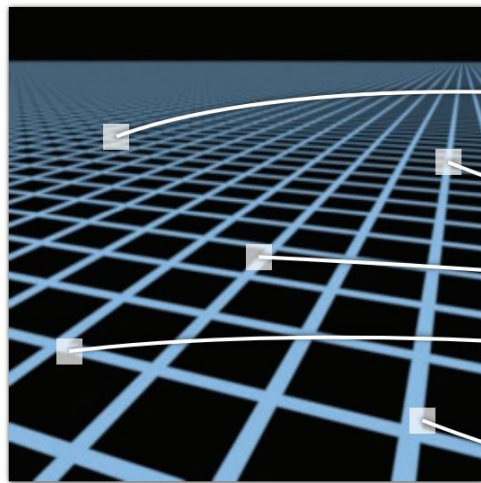


(d)

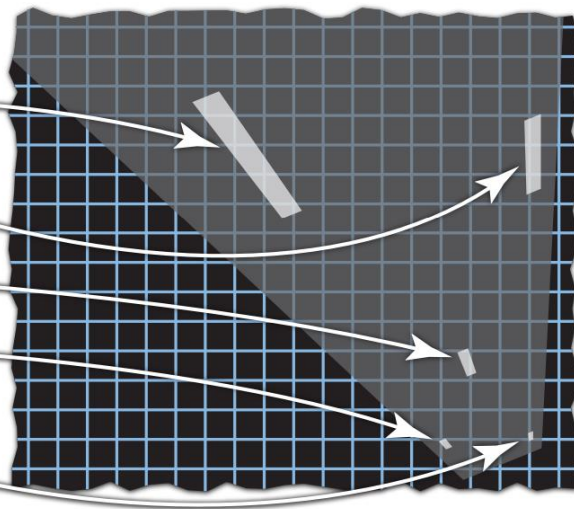
Smooth Mipmap Levels



Anisotropic Mipmaps



Screen space



Texture space



模拟景物表面细节

1

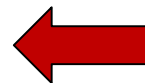
用多边形模拟表面细节

2

纹理的定义和映射

3

凹凸映射

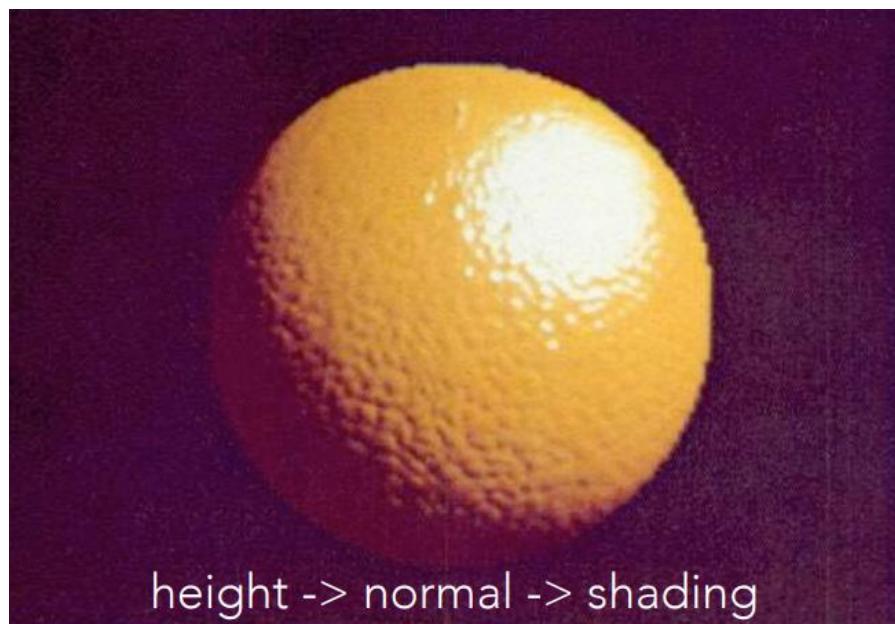


凹凸映射

- 纹理图片可以表示其他信息，比如几何！

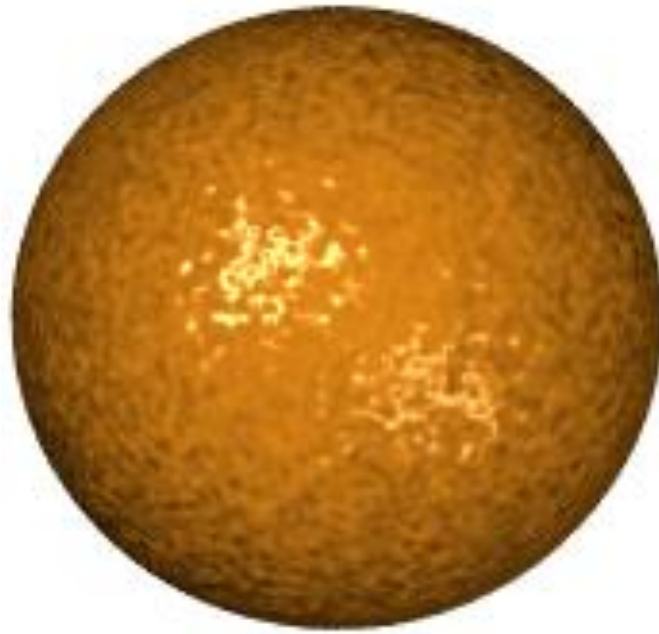
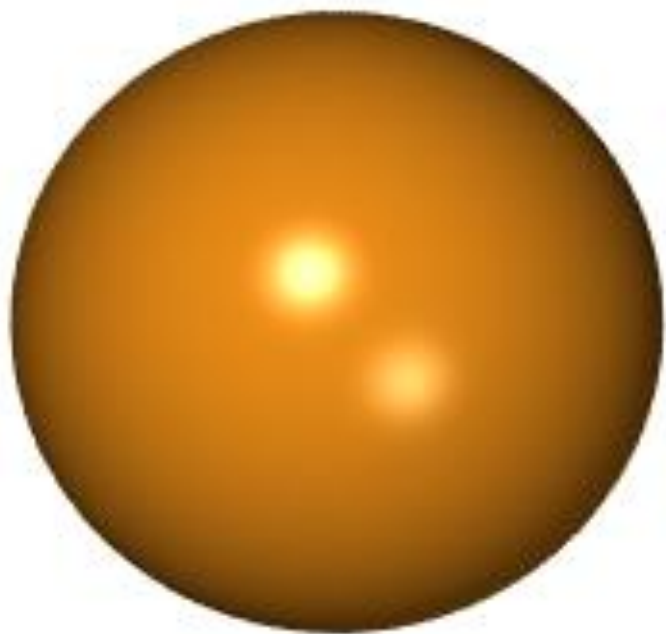


Height Map



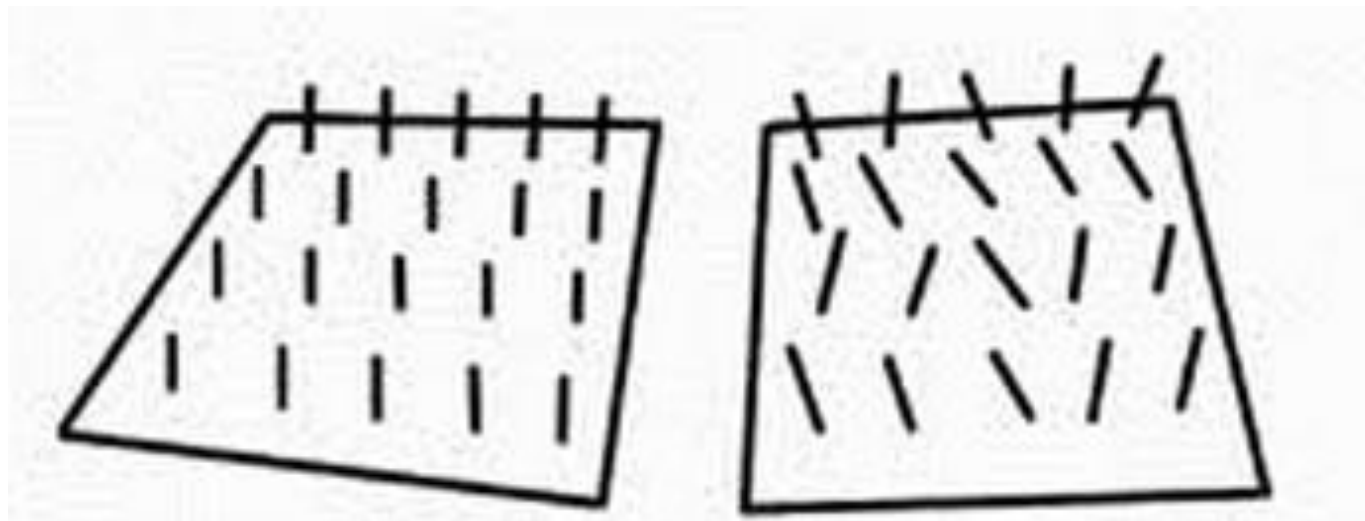
基本想法

- 对曲面的法向进行随机扰动
- 如此得到的图像就会显现出形状变化的错觉



法向

- 曲面上一点的法向量刻画曲面在该点的形状
- 如果用少量扰动曲面法向，那么就会得到具有小变化的曲面
- 如果在生成图像时进行这种扰动，那么就会从光滑的模型得到具有复杂表面模型的图像



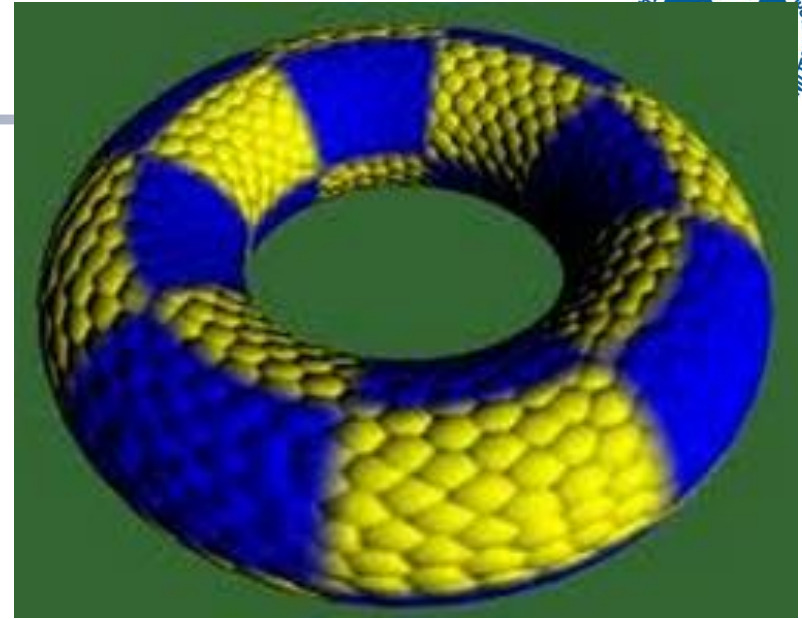
法向扰动方法

- 可以有多种方法对法向进行扰动
- 下述法向适用于参数曲面
 - 设曲面方程为 $S(u,v)$
 - 在一点的单位法向为 $n = S_u \times S_v / \|S_u \times S_v\|$
 - 假设曲面在法向移位 $d(u,v)$ ($|d(u,v)| \ll 1$)
 - 那么变化后的法向近似为
$$n' = n + d_u n \times S_u + d_v n \times S_v$$



结果示例





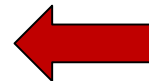
计算方法

- 曲面表示的偏导数计算
 - 基于曲面的表示解析计算，或者
 - 在当前点让 u 与 v 改变很小，找到四个最近点，用差商代替微商
- 位移函数的偏导数计算
 - 根据定义方式解析计算

OpenGL中的光照和纹理

1

点光源



2

全局光照处理

3

表面材质

4

透明处理

5

纹理处理

点光源

- 在OpenGL场景描述中可以包含多个点光源，光源的各种属性设置使用下面的函数指定。

```
void glLight{if} (GLenum light, GLenum pname, TYPE param);
```

```
void glLight{if}v (GLenum light, GLenum pname, TYPE *param);
```

pname取值	默认值	含义
GL_AMBIENT	(0.0, 0.0, 0.0, 1.0)	光源中环境光分量
GL_DIFFUSE	(1.0, 1.0, 1.0, 1.0) 或(0.0, 0.0, 0.0, 1.0)	光源中漫反射光分量
GL_SPECULAR	(1.0, 1.0, 1.0, 1.0) 或(0.0, 0.0, 0.0, 1.0)	光源中镜面光分量
GL_POSITION	(0.0, 0.0, 1.0, 0.0)	光源的坐标位置
GL_SPOT_DIRECTION	(0.0, 0.0, -1.0)	光源聚光灯方向矢量
GL_SPOT_EXPONENT	(0.0)	聚光指数
GL_SPOT_CUTOFF	180.0	聚光截止角
GL_CONSTANT_ATTENUATION	1.0	固定衰减因子
GL_LINEAR_ATTENUATION	0.0	线性衰减因子
GL_QUADRATIC_ATTENUATION	0.0	二次衰减因子

点光源

- 点光源的颜色
- 点光源的位置和类型
- 聚光灯
- 光强度衰减

点光源

- 在OpenGL中，必须明确启用或禁用光照。默认情况下，不启用光照，此时使用当前颜色绘制图形，不进行法线矢量、光源、光照模型、材质属性的相关的计算。要启用光照，可以使用函数：

```
glEnable(GL_LIGHTING);
```

- 指定了光源的参数后，需要使用函数启用light指定的光源：

```
glEnable(light);
```

OpenGL中的光照和纹理

1

点光源

2

全局光照处理

3

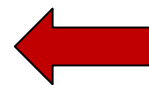
表面材质

4

透明处理

5

纹理处理



□ 在OpenGL中，下面的函数用于指定全局光照

```
void glLightMode{if} (GLenum pname, TYPE param);
```

```
void glLightMode{if}v (GLenum pname, TYPE *param);
```

pname取值	默认值	含义
GL_LIGHT_MODEL_AMBIENT	(0.2, 0.2, 0.2, 1.0)	整个场景的环境光成分
GL_LIGHT_MODEL_LOCAL_VIEWER	GL_FALSE	如何计算镜面反射角
GL_LIGHT_MODEL_TWO_SIDE	GL_FALSE	单面光照还是双面光照
GL_LIGHT_MODEL_COLOR_CONTROL	GL_SINGLE_COLOR	镜面反射颜色是否独立于环境颜色、散射颜色

OpenGL中的光照和纹理

1

点光源

2

全局光照处理

3

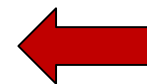
表面材质

4

透明处理

5

纹理处理



OpenGL材质属性

□ 在OpenGL中，下面的函数用于指定材质属性

```
void glMaterial{if} (GLenum face, GLenum pname, TYPE param);
```

```
void glMaterial{if}v (GLenum face, GLenum pname, TYPE *param);
```

pname取值	默认值	含义
GL_AMBIENT	(0.2, 0.2, 0.2, 1.0)	材质对环境光的反射系数
GL_DIFFUSE	(0.8, 0.8, 0.8, 1.0)	材质对漫射光的反射系数
GL_AMBIENT_AND_DIFFUSE		材质对环境光和漫射光的反射系数
GL_SPECULAR	(0.0, 0.0, 0.0, 1.0)	材质对镜面光的反射系统
GL_SHININESS	0.0	镜面反射指数
GL_EMISSION	(0.0, 0.0, 0.1, 1.0)	材质的发射光颜色
GL_COLOR_INDEXES	(0, 1, 1)	环境颜色索引、漫反射颜色索引和 镜面反射颜色索引

OpenGL材质属性

□ OpenGL提供颜色材质模式：

```
glEnable(GL_COLOR_MATERIAL);
```

```
void glColorMaterial (GLenum face, GLenum mode);
```

□ 颜色材质模式中，可以通过glColor函数来指定物体表面的颜色，而相应的材质属性将通过颜色值和光源的RGB值计算出来。

OpenGL中的光照和纹理

1

点光源

2

全局光照处理

3

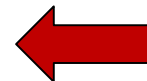
表面材质

4

透明处理

5

纹理处理



透明处理

- **OpenGL中使用混合实现透明处理。**
- **混合操作是指将输入对象（源）的颜色值与当前存储在帧缓存中的像素（目标）颜色值合并的过程。**

透明处理

□ 开启混合操作

```
glEnable(GL_BLEND);
```

□ 指定计算源因子和目标因子的计算方式

```
void glBlendFunc(GLenum srcfactor, GLenum  
destfactor);
```

常量	RGB混合因子	alpha混合因子
GL_ZERO	(0, 0, 0)	0
GL_ONE	(1, 1, 1)	1
GL_SRC_COLOR	(R_s, G_s, B_s)	A_s
GL_ONE_MINUS_SRC_COLOR	$(1, 1, 1) - (R_s, G_s, B_s)$	$1 - A_s$
GL_DST_COLOR	(R_d, G_d, B_d)	A_d
GL_ONE_MINUS_DST_COLOR	$(1, 1, 1) - (R_d, G_d, B_d)$	$1 - A_d$
GL_SRC_ALPHA	(A_s, A_s, A_s)	A_s
GL_ONE_MINUS_SRC_ALPHA	$(1, 1, 1) - (A_s, A_s, A_s)$	$1 - A_s$
GL_DST_ALPHA	(A_d, A_d, A_d)	A_d
GL_ONE_MINUS_DST_ALPHA	$(1, 1, 1) - (A_d, A_d, A_d)$	$1 - A_d$
GL_CONSTANT_COLOR	(R_c, G_c, B_c)	A_c
GL_ONE_MINUS_CONSTANT_COLOR	$(1, 1, 1) - (R_c, G_c, B_c)$	$1 - A_c$
GL_CONSTANT_ALPHA	(A_c, A_c, A_c)	A_c
GL_ONE_MINUS_CONSTANT_ALPHA	$(1, 1, 1) - (A_c, A_c, A_c)$	$1 - A_c$
GL_SRC_ALPHA_SATURATE	$(f, f, f); f = \min(A_s, 1 - A_d)$	1

透明处理

- 进行混合计算
- 假定计算出的源和目标混合因子分别为 (S_r, S_g, S_b, S_a) 和 (D_r, D_g, D_b, D_a) ，并且分别使用下标 s 和 d 区分表示源和目标的 $RGBA$ 值，则混合后的 $RGBA$ 值如下：

$$(R_s S_r + R_d D_r, G_s S_g + G_d D_g, B_s S_b + B_d D_b, A_s S_a + A_d D_a)$$

OpenGL中的光照和纹理

1

点光源

2

全局光照处理

3

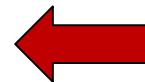
表面材质

4

透明处理

5

纹理处理



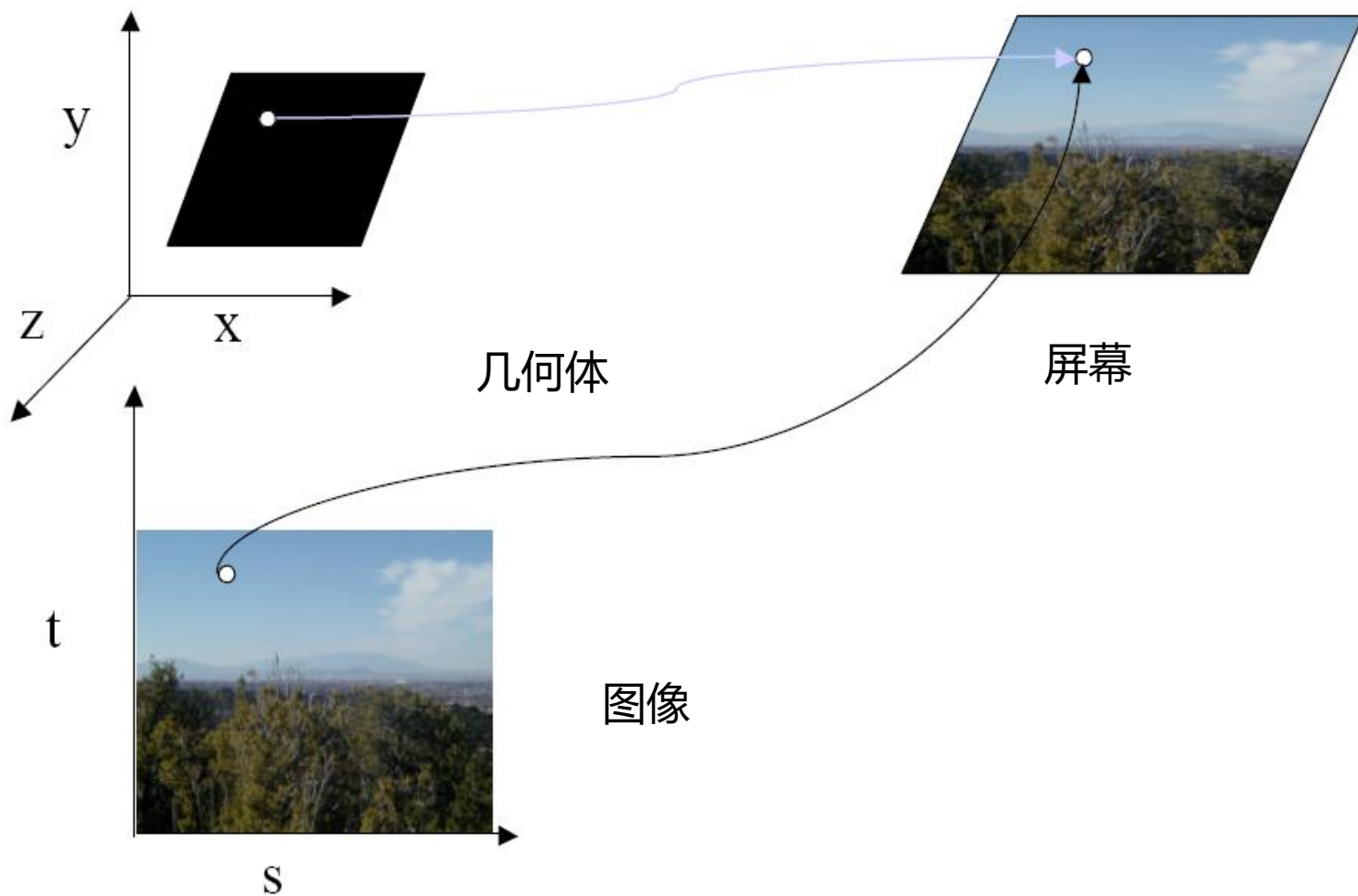
OpenGL与纹理

- OpenGL支持许多纹理映射选项
 - 第一版包含了把一维或二维纹理映射到一维至四维图形对象的函数
 - 现在的版本提供了三维纹理，但需要高端硬件的支持

基本策略

- 应用纹理需要下面三个步骤
 - 指定纹理
 - 读入或生成图像
 - 赋给纹理
 - 激活纹理映射功能
 - 给每个顶点赋纹理坐标
 - 由应用程序建立适当的映射函数
 - 指定纹理参数
 - 包装 (wrapping) , 过滤 (filtering)

纹理映射



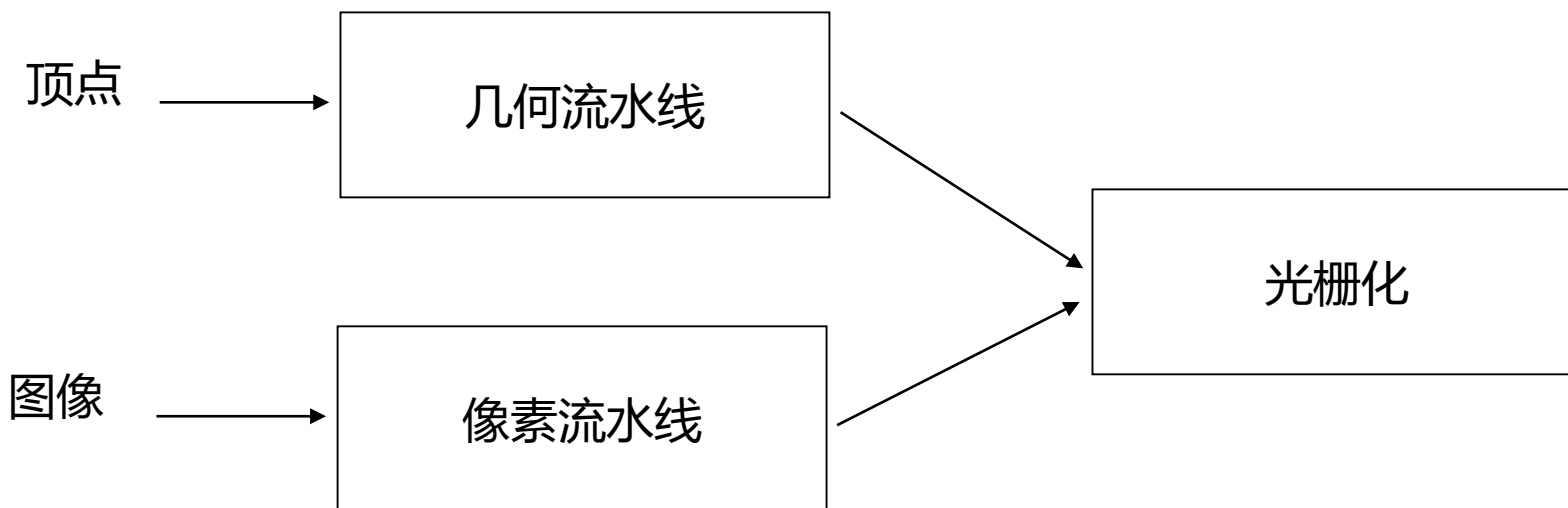
纹理示例

- 纹理（下方）是 256×256 的图像，它被映射到一个矩形上，经透视投影后的结果显示在上方



纹理映射与OpenGL流水线

- 图像与几何分别经过不同的流水线，在光栅化时合二为一
 - 复杂纹理并不影响几何的复杂性



指定纹理图像

- 利用CPU内存中的纹理元素数组定义纹理图像
 - `GLubyte my_texels[512][512];`
- 定义纹理图像所用的像素图
 - 扫描图像
 - 由应用程序代码创建
- 激活纹理映射
 - `glEnable(GL_TEXTURE_2D);`
 - OpenGL支持一至四维纹理映射

定义纹理所用的图像

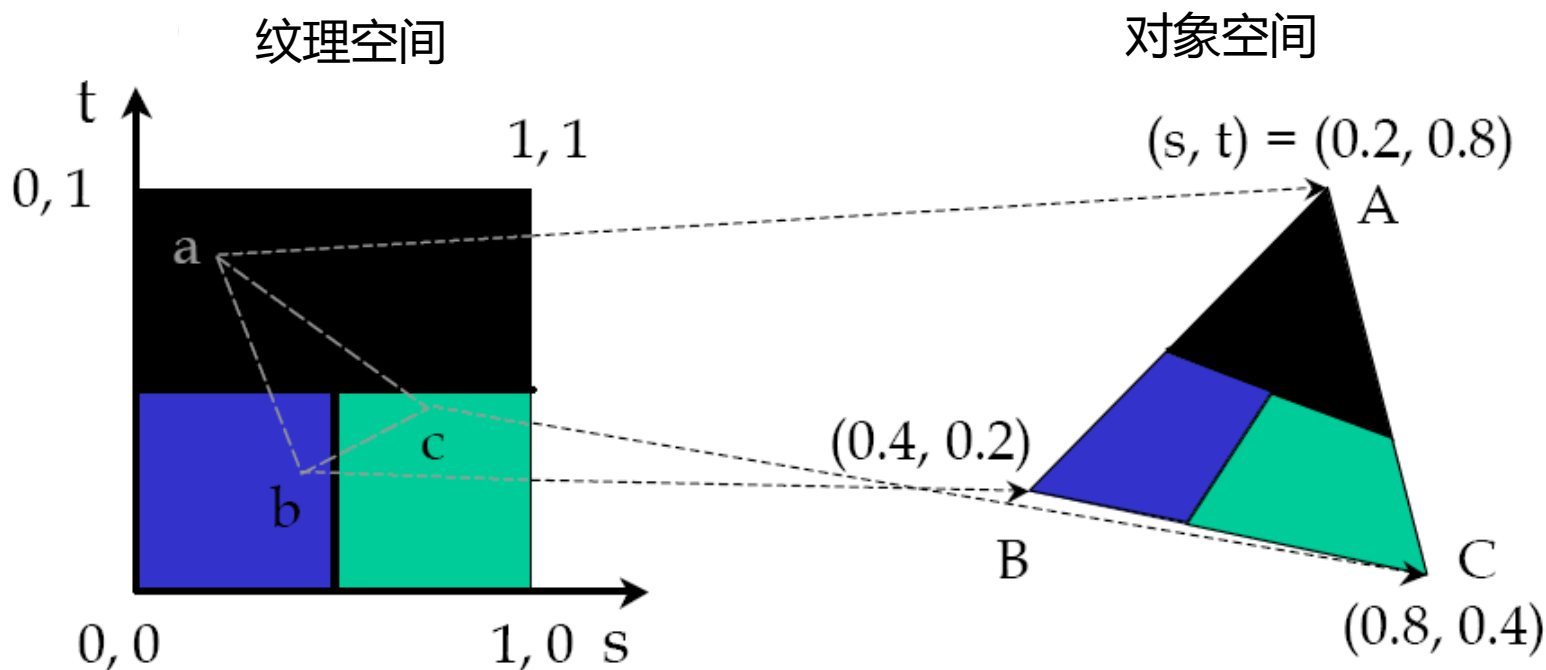
- `glTexImage2D(target,level,components,w,h,border,format,type,texels);`
target:纹理的类型，例如：GL_TEXTURE_2D
level:用于mipmapping(稍后讨论)
components:每个纹理元素的分量数
w,h:texels中以像素为单位的宽度与高度
border:用于光滑处理(稍后讨论)
format与type:描述纹理元素
texels:指向纹理元素数组的指针
- `glTexImage2D(GL_TEXTURE_2D,0,3,512,512,0,GL_RGB,GL_UNSIGNED_BYTE,my_texels);`

转化纹理图像

- OpenGL需要纹理的尺寸为2的幂次
- 如果图像的尺寸不是2的幂次，用下述函数进行转化
 - **gluScaleImage(format,w_in,h_in, type_in,*data_in,w_out,h_out, type_out,*data_out);**
 - data_in 源图像
 - data_out 目标图像
- 当放缩时图像被插值和过滤

映射纹理

- 基于参数纹理坐标
- `glTexCoord* ()` 指定每个顶点对应的纹理坐标



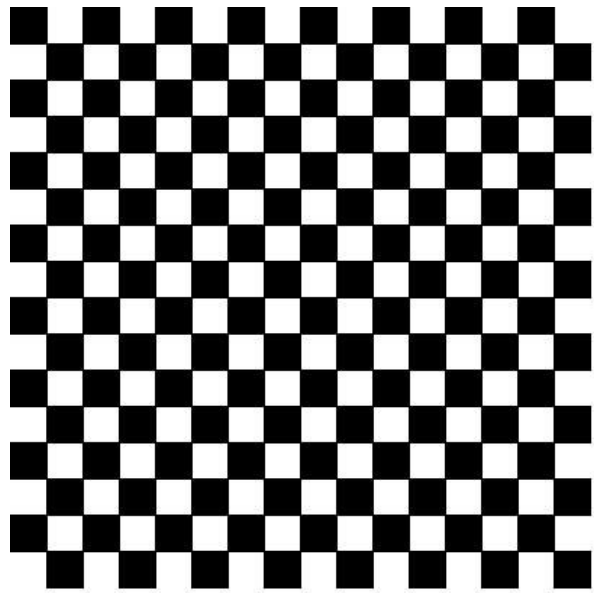
典型代码

- glBegin(GL_POLYGON);
 glColor3f(r0,g0,b0);
 glNormal3f(u0,v0,w0);
 glTexCoord2f(s0,t0);
 glVertex3f(x0,y0,z0);
 glColor3f(r1,g1,b1);
 glNormal3f(u1,v1,w1);
 glTexCoord2f(s1,t1);
 glVertex3f(x1,y1,z1);
 .
 .
 glEnd();

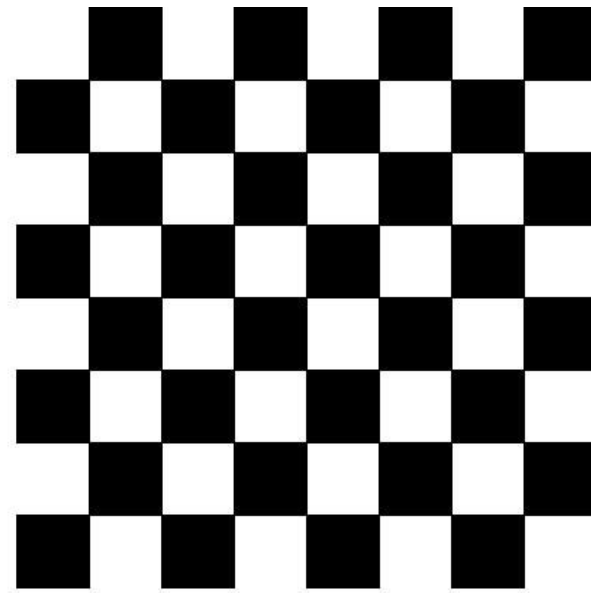
注意为了提高效率，可以采用顶点数组

插值

- OpenGL应用双线性插值从给定的纹理坐标中求出适当的纹理元素
- 可以只应用纹理的一部分
 - 方法是只应用纹理坐标的一部分，如最大纹理坐标为 $(0.5, 0.5)$



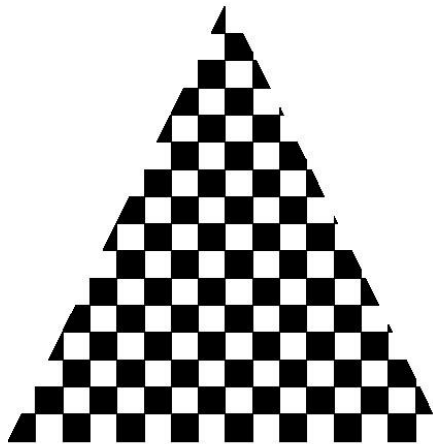
(a)



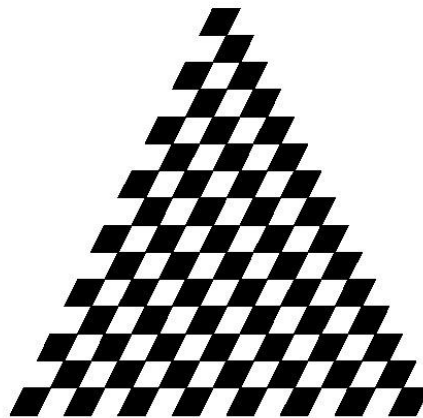
(b)

变形

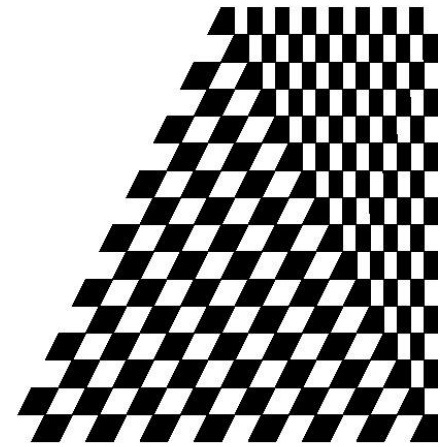
- 对于四边形，从纹理坐标到纹理元素的对应是比较直接的
- 对于一般的多边形，OpenGL需要确定一种方法，确定纹理坐标与纹理元素的对应
 - 可能会出现变形



(a)



(b)



(c)

纹理参数

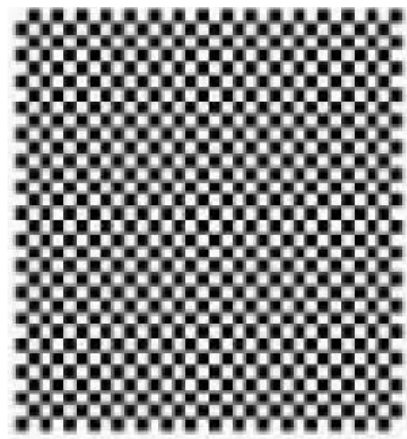
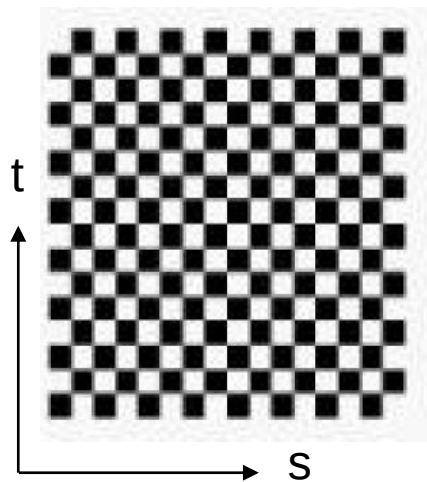
- OpenGL中有许多办法确定纹理的使用方式
 - Wrapping参数确定当s, t的值超出[0,1]区间后的处理方法
 - 应用filter模式就会不采用点取样方法, 而是采用区域平均方法
 - Mipmaping技术使得能以不同的分辨率应用纹理
 - 环境参数确定纹理映射与明暗处理的交互作用

Wrapping模式

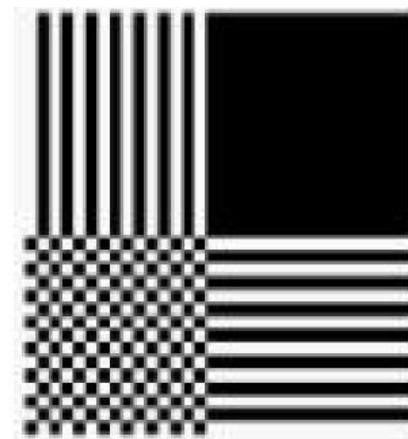
- 截断：若 $s, t > 1$ 就取1，若 $s, t < 0$ 就取0
- 重复：应用 s, t 模1的值

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP)
```

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT)
```



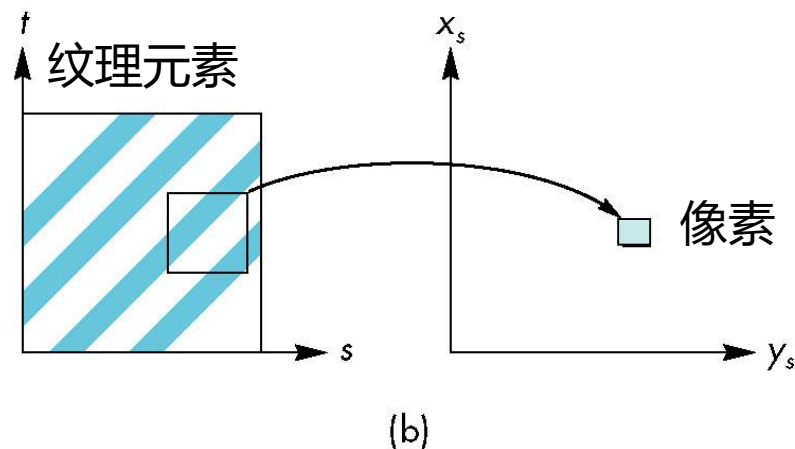
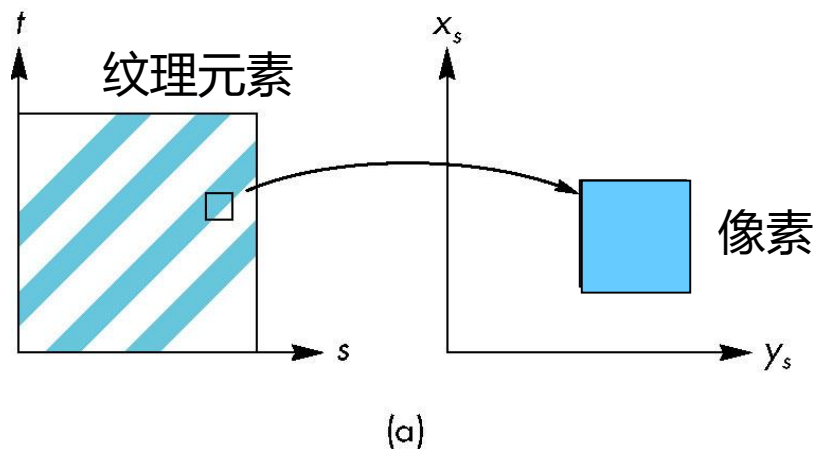
GL_REPEAT



GL_CLAMP

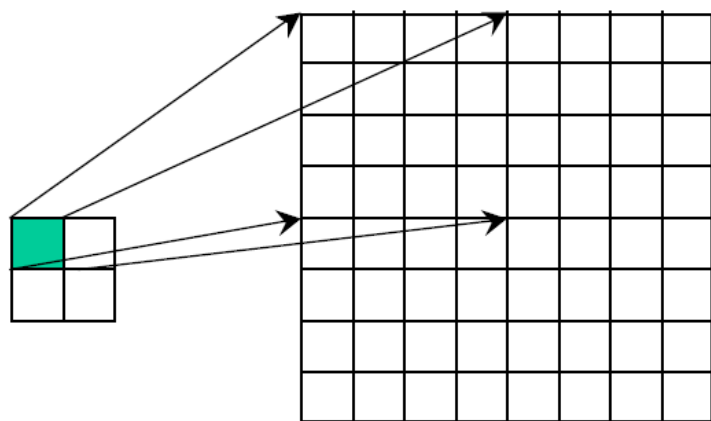
放大与缩小

- 可以是多个纹理元素覆盖一个像素（缩小），也可以是多个像素覆盖一个纹理元素（放大）



解决方法

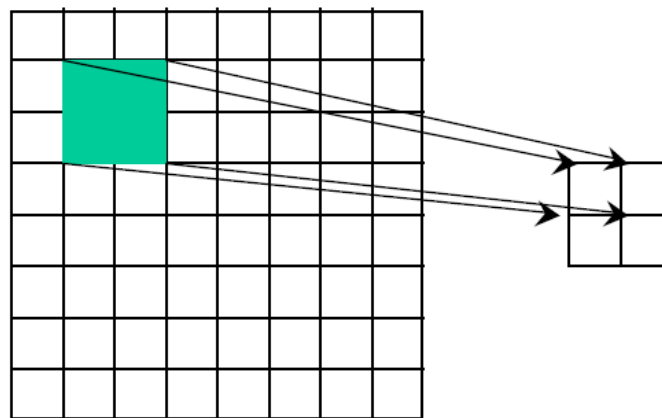
- 可以应用点取样（最近纹理元素）或者线性滤波（2x2滤波）得到纹理值



纹理

多边形

放大



纹理

多边形

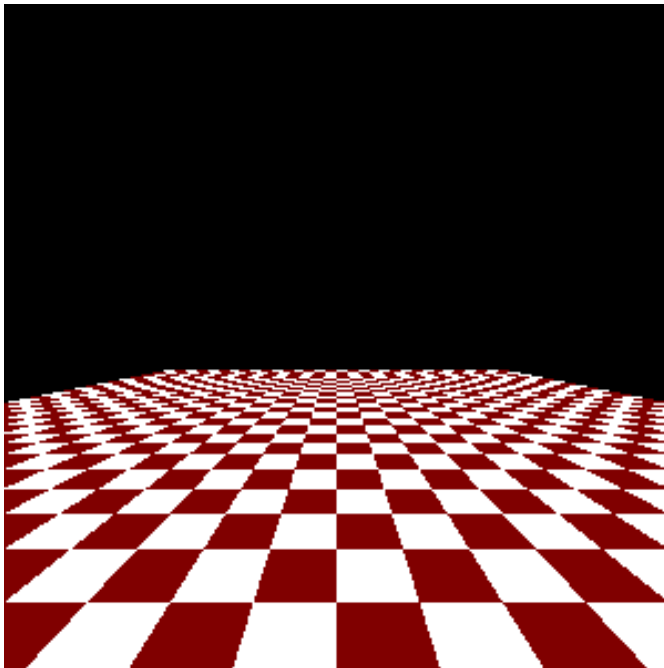
缩小

滤波模式

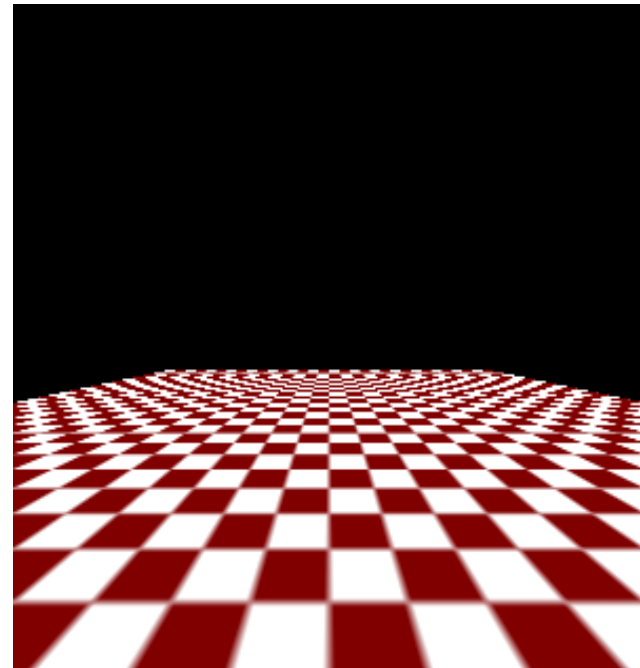
- 模式指定
 - `glTexParameteri(target, type, mode)`
 - `glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);`
 - `glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);`
- 注意在线性滤波中为滤波边界需要纹理元素具有额外的边界 (`border=1`)

Mipmap

- 在纹理定义时声明mipmap的层次
 - `glTexImage2D(GL_TEXTURE_2D, level, ...);`
- GLU中提供了mipmap建立界面，可以从给定图像建立起所有的mipmap纹理
 - `gluBuild2DMipmaps(...)`



GL_NEAREST



GL_LINEAR

纹理函数

- 控制纹理的应用方式
 - `glTexEnv{fi}[v](GL_TEXTURE_ENV, mode, param);`
 - `GL_TEXTURE_ENV_MODE` 设置模式
 - `GL_MODULATE`: 与计算的明暗效果调制在一起
 - `GL_BLEND`: 与环境颜色融合在一起
 - `GL_REPLACE`: 只应用纹理的颜色
 - `GL_DECAL`: 与`GL_REPLACE`相同
 - 应用`GL_TEXTURE_ENV_COLOR`设置融合颜色

透视校正提示

- 纹理坐标与颜色插值
 - 要么是关于屏幕空间线性确定的
 - 或者要应用深度/透视值（较慢）
- 对于在边界上的多边形，效果非常明显
 - **glHint(GL_PERSPECTIVE_CORRECTION_HINT, hint)**
其中hint可取值GL_DONT_CARE, GL_NICEST, GL_FASTEST

纹理坐标的生成

- OpenGL可以自动生成纹理坐标
 - **glTexGen{ifd}[v](GLenum c, GLenum pn, pv)**
 - **c**: 纹理坐标, 可取GL_S, GL_T, GL_R, GL_Q
 - **pn**: 纹理坐标的生成函数, 可取
GL_TEXTURE_GEN_MODE, GL_OBJECT_PLANE,
GL_EYE_PLANE
 - **pv**: 参数值, 当pn=GL_TEXTURE_GEN_MODE时, 可取
GL_OBJECT_LINEAR, GL_EYE_LINEAR,
GL_SPHERE_MAP; 否则应为数组, 由生成函数的系数组成

纹理对象

- 纹理也是状态的一部分
 - 如果不同的对象具有不同的纹理，那么OpenGL需要从处理器内存向纹理内存传送大量数据
- 最新版本的OpenGL提供了纹理对象功能
 - 每个纹理对象是一个图像
 - 纹理内存可以保存多个纹理对象

谢谢！

*Thank
You*